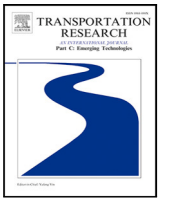
Transportation Research Part C 179 (2025) 105278



Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc





Generating practical last-mile delivery routes using a data-informed insertion heuristic

Hesam Rashidi [a], Mehdi Nourinejad [b,*], Matthew Roorda [a]

[a] Department of Civil & Mineral Engineering, University of Toronto, Toronto, Canada
[b] Department of Civil Engineering, York University, Toronto, Canada

ARTICLE INFO

Keywords:
Last-mile delivery
Data-informed routing
Graph neural networks
Classification
Insertion heuristic

ABSTRACT

Couriers often deviate from pre-planned delivery routes due to practical realities that routing algorithms may overlook. We statistically demonstrate that, in addition to travel time, factors like turn sharpness, backtracking distance, and neighbourhood visit timing influence a driver's navigational choices and propose a Data-informed Insertion Heuristic (DIIH) for Travelling Salesman Problems (TSPs), which considers a custom cost function inferred from historical routes. The DIIH is trained on the dataset from Amazon's Last-mile Research Challenge, which contains historical TSP instances classified into one of three qualities: high, medium, or low, indicating the satisfaction level of Amazon's logistics planners of a route based on productivity, courier experience, and customer satisfaction levels. We train an energy-based model to predict the likelihood of a route being of high quality. Compared to existing benchmarks, the DIIH generates 22.4% and 24.1% additional high-quality solutions than the Amazon challenge winner and courier-performed routes, respectively. This improvement comes with an increase of 20.6% and 13.9% in the median travel time. While optimizing purely for travel time would result in shorter routes, we account for both travel time and human preferences, which explains the observed tradeoff. We show that the probabilistic evaluation of a route measured by the energy-based model developed in this study is a promising metric for estimating a routing algorithm's practical performance.

1. Introduction

Last-mile delivery is the final leg of the supply chain, where trucks transport goods from a depot to end consumers. Last-mile routing is a combinatorial optimization problem usually formulated as the Travelling Salesman Problem (TSP) (Dantzig and Ramser, 1959) for a single vehicle and as the Vehicle Routing Problem for multiple vehicles. The classical TSP generates the optimal route (e.g., route with minimum total travel time) that starts from a depot and visits each recipient exactly once before returning to the depot. While TSP routes may be efficient on paper in terms of minimizing total delivery time, empirical evidence suggests that they are often impractical, and drivers frequently *deviate* from them (Li and Phillips, 2018; Merchán et al., 2022). Deviations occur as the routing algorithms do not consider driver preferences and experiences (Holland et al., 2017; Özarık et al., 2023). Drivers progressively develop experience from repeated deliveries in the same regions, which helps them create more practical routes than conventional TSPs (Rastogi, 2023; Ulmer et al., 2020); for instance, experienced drivers may have knowledge of local traffic patterns, recipient availability, or convenient stop locations (Quirion-Blais and Chen, 2021; Dia, 2002). This study develops a

* Corresponding author.
E-mail addresses: hesam.rashidi@mail.utoronto.ca (H. Rashidi), mehdi.nourinejad@lassonde.yorku.ca (M. Nourinejad), matt.roorda@utoronto.ca (M. Roorda).

https://doi.org/10.1016/j.trc.2025.105278
Received 22 November 2024; Received in revised form 9 July 2025; Accepted 11 July 2025
Available online 25 July 2025
0968-090X/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

learning framework that extracts latent driver experiences from historical delivery routes and exploits them within a data-informed heuristic to generate more practical delivery routes.

The practical limitations of conventional TSPs stem primarily from two areas. The first is related to tractability, as conventional TSPs optimize routes by making some simplifying assumptions (e.g., assuming static travel times). Routing algorithms choose faster, albeit less precise, solutions that satisfy recipient expectations for timely deliveries. These simplifying assumptions, however, pose a trade-off against practical performance. For instance, considering stochastic travel times in routing algorithms enhances solution practicality but adds more complexity (Bai et al., 2021; Zhou et al., 2019). The second reason limitation of conventional TSPs is that they often omit real-world factors, like recipient availability, which are challenging to model and predict. The lack of such information can result in inefficient routes and increased operational costs due to failed deliveries (Florio et al., 2018; Özarık et al., 2021). Incorporating these factors, although nontrivial, could lead to higher-quality routes.

Recent literature suggests that *human-centred* algorithms enhance the practicality of operational planning. A human-centred algorithm leverages the workforce's tacit knowledge and accumulated daily experience to improve productivity and service quality. By leveraging the workforce's expertise, these refined algorithms align more closely with the real world, ultimately leading to solutions that adhere to both human preferences and operational efficiency. For instance, Sun et al. (2022) investigated how warehouse workers at Alibaba often deviate from prescribed algorithmic instructions (i.e., from conventional packing algorithms) on how to pack orders within a box to optimize space utilization. The study identified two main types of deviations: *information deviations*, where workers had better information than the algorithm, and *complexity deviations*, where workers found the instructions impractical and too complex to follow. Sun et al. (2022) developed a human-centric packing algorithm to accommodate human deviations. Their field test showed that their algorithm reduced packing time for *targeted* packages (i.e., packages with instructions that the workers will most likely deviate from) by 4.5% and would save an average of 2.6 million USD annually if applied to all of Alibaba's warehouses. Similarly, Gattermann-Itschert et al. (2023) enhanced crew scheduling algorithms for railways by incorporating planners' subjective preferences. The study trained a Machine Learning (ML) classifier on over 16,000 instances of planner feedback to predict the likelihood of planners marking a duty as undesirable. Gattermann-Itschert et al. (2023) incorporated the ML classifier into the scheduling algorithm to generate more practical schedules. Their method increased the likelihood of employee schedule conformance by an average of 12% while increasing operational costs by less than 1%. Our study, analogous to the aforementioned studies, develops a human-centric routing algorithm for last-mile deliveries. We leverage historical data to discern practical delivery route structures and to capture the implicit knowledge of expert drivers regarding the best times to visit neighbourhoods within a service region.

This study presents a two-phase framework for generating solutions to TSPs with soft time windows (i.e., early and late arrivals are allowed but penalized). The framework learns from driver experiences in the offline phase (i.e., the *learning* phase) and generates solutions to newly realized instances in the online phase (i.e., the *inference* phase). We train the framework on the open-source dataset from Amazon's *Last-mile Research Challenge* (Merchán et al., 2022). The dataset includes over 6000 historically realized TSP instances in the United States. For each TSP instance, the actual sequence in which the driver visited the customers is documented and classified into one of three quality classes: high, medium, or low. The route quality labels indicate the satisfaction level of logistics planners regarding a given observed route. These labels are based on a route's productivity, the driver experiences, and customer satisfaction levels (Merchán et al., 2022). Our framework uses supervised ML to identify patterns between the structural characteristics of a route and its quality class. We use the insights from the ML model to create a Data-informed Insertion Heuristic (DIIH) (Campbell and Savelsbergh, 2004). The DIIH generates solutions that conform to the constraints of a TSP, have a high efficiency on paper, and have great potential to be of high quality in the real world. Inspired by Ben-Akiva et al. (2004), Özarık et al. (2023), and Reyes et al. (2017) the DIIH generates a set of alternative TSP tours (subject to a runtime limit) and then prescribes the tour with the best quality as the final solution. This increases the heuristic's chance of finding diverse solutions and escaping local optima. Our heuristic, however, differs from past studies in how it *creates* the set of alternative routes and how it *selects* the best route (refer to Section 4.4 for more information).

We make several contributions to the literature. First, we show statistical evidence that route quality is not merely measured by how quickly packages are delivered. Other route features, like turn sharpness, backtracks to visited neighbourhoods, and the choice of visiting times to neighbourhoods, affect route quality. We propose a new metric that captures a route's overall quality of neighbourhood visit times in a low-dimensional graph embedding. We determine the quality of a neighbourhood's time of visit by analysing how frequently a particular time slot for visiting the neighbourhood has been included in past high-quality routes and the success rate of deliveries during the time slot. We show that our proposed metric learns from drivers' past visit time choices and can transfer their latent knowledge and expertise to new fleet drivers. This transfer of knowledge assists new drivers in learning how to plan their routes, improving overall efficiency and successful delivery handoff. Next, we propose an energy-based model for evaluating the practical performance of a routing algorithm. The literature generally uses metrics based on Edit Distance (Mccoy and Frank, 2018; Dieter et al., 2023), which compares the visit sequence of a TSP tour to a *desired* tour. Similarly, Amazon's challenge defined a weighted disparity metric that compares a proposed visit sequence to a known high-quality visit sequence. The metric aggregates the cost (based on the normalized travel time matrix) and the number of edits required to transform the proposed tour to the desired. While the disparity metric is useful for comparing routing algorithms in a competition setting, it does not quantify the performance quality of an algorithm. The output of the disparity metric does not directly predict whether a tour will be of high quality in the field. If the goal is to generate more high-quality solutions within a routing algorithm, we need a metric that predicts the quality of a route. We address this limitation by proposing an energy-based model that predicts the route quality using factors such as the instance structure (e.g., number of stops and packages), route duration, backtracking distance, turn sharpness, and neighbourhood visit times. Lastly, the proposed framework can be applied to other TSP variations. With minor modifications

to the DIIH's structure, for instance, the framework may be applied to a TSP with pick-ups and deliveries (e.g., on-demand food delivery service). Alternatively, the DIIH supports TSPs with dynamic order realization as it can insert stops to a partial route.

The remainder of this paper is organized as follows. Section 2 reviews the past research on data-driven last-mile routing and highlights this study's key differences from the literature. In Section 3, the data source and its content are introduced. Section 4 explains the two-phase framework developed in this study in detail. In Section 5, we explain the benchmarks, evaluation metrics, and the computing setup used in this study. Section 6 reports the results from the framework and comments on the observed performances. Finally, in Section 7, we summarize the findings of this study and present potential avenues for future research.

2. Related work

We identify two streams of relevant research. (1) Consistent Vehicle Routing Problems, a modified formulation of the TSP that prioritizes driver-service region familiarity in routing, and (2) behaviour-driven routing - studies that use learning methods to integrate drivers' past navigational choices into last-mile routing. The following reviews the most relevant studies using the Amazon dataset, however, we refer the readers to the work of Winkenbach et al. (2024) for a detailed review of the algorithms that competed in the Amazon challenge.

2.1. Consistent vehicle routing

The Consistent Vehicle Routing Problem (ConVRP) is a variant of the TSP used for human-centred planning for last-mile deliveries (Kovacs et al., 2015a; Ulmer et al., 2020). The ConVRP maintains consistency in the driver assigned to a particular geographical region or customer, which empowers drivers to plan their delivery schedules and use their tacit experience (Schneider, 2016). Regular delivery schedules give customers the predictability they need to plan their day, and having the same drivers perform the deliveries can establish strong customer relationships (Luo et al., 2015).

Kovacs et al. (2015a,b) studied whether there exists an efficiency tradeoff between temporal consistency, driver consistency, and optimal routing. They showed that having at least two drivers familiar with a particular region reduces operational costs. Similarly, Luo et al. (2015) formulated a Mixed Integer Program to serve customers by a consistent limited group of drivers. They demonstrated that driver consistency reduces operation costs under relatively stable demand patterns. Ulmer et al. (2020) studied the long-term benefits of driver-customer familiarity in deliveries. They formulated the problem as a Markov Decision Process, defining familiarity as a binary variable indicating whether the driver has previously visited a customer. Their results demonstrate that investing in increasing and maintaining driver familiarity, as opposed to solely focusing on optimal short-term routing, results in long-term cost savings.

The challenge with the ConVRP is that driver knowledge remains latent to the routing algorithm. Hence, when drivers accustomed to serving a specific zone leave the fleet, or new drivers are expected to make deliveries frequently (especially in the gig economy), the company must navigate the learning curve new drivers face to handle deliveries efficiently. This transition period introduces inefficiencies in the delivery service. This study develops a framework that *learns* from the driver experiences and integrates them into planning, thus facilitating the process of familiarizing the new drivers with the service region.

2.2. Learning driver behaviour

Inverse Optimization. Inverse optimization is a mathematical technique that determines the parameters of an optimization model based on observed decisions rather than formulating an objective function and constraints to derive an optimal solution. Bärmann et al. (2018) designed an online learning framework that used inverse optimization to learn the cost matrix in a simple delivery network. The study argues that drivers know the *true* cost matrix of the system, and use drivers' routing choices to calibrate its cost matrix over time. Chen et al. (2021) extended the work of Bärmann et al. (2018) to a VRP with capacity constraints.

Zattoni Scroccaro et al. (2025) and Özarık et al. (2023) used inverse optimization to tackle the Amazon challenge. Zattoni Scroccaro et al. (2025) proposed a methodology to learn a custom weight matrix using inverse optimization to approximately regenerate observed courier-performed routes. They modify the travel distance matrix based on couriers' learned preferred zone visit sequences and create a feasible tour based on the matrix. Özarık et al. (2023) developed an inverse optimization framework called `Pool & Select`. For a new TSP instance, they create a *pool* of potential solutions and *select* the solution with the highest predicted disparity score (i.e., an estimation of the solution's disparity score in the Amazon challenge). Özarık et al. (2023) generate the pool of solutions using a generic Random Insertion heuristic (Rosenkrantz et al., 1977). They penalize backtracks and bias the heuristic to generate routes with similar entry/exit neighbourhoods as the routes within the historical data. They estimate a solution's disparity score using an ML regression model trained on high-quality solutions present in the dataset.

Similar to the approach developed by Özarık et al. (2023), Nourmohammadi et al. (2025) developed an ML framework to learn the implicit preferences of route planners, which indirectly captures the preferences of drivers and customers from historical delivery tours from an Australian courier service. They define an objective function that optimizes three terms: the distance travelled by the delivery vehicles, the number of vehicles dispatched, and the balance of driver workload across the fleet. They select the parameters for these three terms based on a weighted sampling strategy combined with supervised learning.

Inverse optimization is limited in learning from routes with favourable characteristics (i.e., high-quality routes). Inverse optimization assumes that the observed solutions are optimal for the given problem instances (Chan et al., 2018). As a result,

if an observed route does not exhibit high quality, it cannot be incorporated into an inverse optimization framework. This restricts the possibility of deriving insights and identifying patterns to *avoid* future solutions to the problem.

Adaptive Memory. The underlying premise of Adaptive Memory (AM) is that superior solutions to a combinatorial problem are likely to contain frequently appearing components in lower-quality solutions of the problem (Rochat and Taillard, 1995). The AM methodology solves a combinatorial problem over time and refines its pool of previously generated solutions. Using the AM framework, Quirion-Blais and Chen (2021) introduced a novel approach by incorporating driver experiences into last-mile planning. The method retrieved the most similar observed instances to a new instance and formalized them to construct a new plan. They retrieved similar historical sequences, pruned and merged them to build a new delivery plan, revised it to avoid constraint violations, and retained the final plan in their repository (i.e., the historical pool of sequences) based on a ranking function.

Cook et al. (2022) developed a constrained local search meta-heuristic that extracted the visit order sequence of the *high quality + delivered* instances in the Amazon dataset (i.e., instances where the driver's route was high-quality, and all delivery attempts were successful) and penalized straying away from these sequences in the solution generation process. While their proposed method ranks first in Amazon's competition, it only tries to replicate *favourable* delivery routes using a small subset of the available dataset (i.e., 1107 out of 6112 historical instances). The remaining subset can generate additional insight into the inherent patterns in deliveries. A learning framework has the potential to learn patterns that lead to both favourable (i.e., high quality + delivered) and unfavourable (i.e., high quality + failed, medium, and low quality) routes and integrate them into planning accordingly.

Learning Transition Preferences. Behaviour-driven routing can focus on learning *transition preferences*—that is, how likely or desirable it is to move from one location (or zone) to another, based on historical patterns. One classical probabilistic approach to capturing such preferences is Markov modelling. Krumm (2008) developed a Markov model to predict driver turning behaviour based on near-term observed driven segments, achieving a 90% accuracy rate in predicting the next road segment. Ye et al. (2015) used Markov models to determine a driver's entire route, given their historical choices and a specific origin and destination. Canoy and Guns (2019) applied first-order Markov models to reproduce drivers' routing preferences in last-mile delivery services and used zone transition probabilities to solve a maximum likelihood VRP. Canoy et al. (2023) extended this framework by exploring the use of second-order Markov models to determine transition probabilities. Similarly, Canoy et al. (2024) used a first-order Markov model to learn the zone-level transition probabilities from historical data to solve a TSP, including the zone-level sequences as a penalty in the objective function.

An alternative to probabilistic transition modelling is to embed preferences directly into a cost matrix, modifying the arc costs to reflect historically favourable or unfavourable transitions. Sun and Arslan (2025) proposed a contextual optimization framework that learns a feature-weighted transition matrix from high-quality routes. Their method combines rule-based and data-driven learning to penalize undesirable transitions and encourage routes that resemble high-quality routes.

3. Amazon's last-mile routing challenge dataset

3.1. Dataset structure

In 2021, Amazon partnered with the Massachusetts Institute of Technology (MIT) to host the last-mile routing challenge. The goal of the challenge was to use Artificial Intelligence (AI) to reduce the quality gap between the routes executed by Amazon delivery drivers and the routes generated by conventional TSPs, by learning from real-world routing data. The competition's data is publicly available and includes 6112 TSP instances across five cities in the United States (i.e., Austin, Boston, Chicago, Los Angeles, and Seattle). The logistics planners at Amazon have classified each realized delivery route (i.e., route taken by drivers) in the dataset into one of three qualities: high, medium, or low. "These labels were created by analysing route quality attributes and delivery defects along with various other dimensions of interest. These dimensions of interest can broadly be categorized as related to driver experience, customer satisfaction, and productivity" (Merchán et al., 2022). Amazon ranked the competing methods based on a disparity score measuring how closely they matched the sequences in unseen high-quality routes (see Section 5.2).

The dataset contains three levels of information for each realized route: package-level, stop-level, and route-level data (Merchán et al., 2022). Each level is explained in the following:

- The **package-level** information focuses on the parcels, detailing their spatial dimensions, whether the driver's delivery attempt was successful, the assigned delivery time windows for the packages, and the expected service time for each package. There are 3449 routes with at least one unsuccessful initial delivery attempt. In 859 routes, the drivers retried delivery for at least one package. Over 30% of these routes have had more than one package delivery reattempt, with a single route having delivery retries for 18 packages. Less than 8% of packages have specified delivery time windows in the dataset. For stops assigned with multiple packages, this study selects the most restrictive time window among the packages for the entire stop. That is, the latest start time and earliest end time among the packages' time windows are taken for the stop.
- The **stop-level** data include the geographical coordinates of each stop, the list of packages designated for delivery at these locations, the geographical planning area for each stop, and the average historical travel times between pairs of stops.
  Amazon has adopted a system for categorizing geographical planning areas, associating a zone ID to each stop in the format of `A-2.3B`. Inspired by Özarık et al. (2023), we extract four levels of zones from the zone-IDs: Supra, Inter, Intra, and Loci zones. At the broadest level, the supra zones are defined by appending the city name from which a delivery route originates to a unique identifier. For instance, `LA+A` denotes a supra zone, where `LA` identifies the city and `A` signifies a specific area within the broader region serviced by that depot. The inter zone introduces a numerical component to refine the geographical

Table 1
Descriptive statistics of the dataset. The table reports the average and standard deviation of each variable.

Information Level	Variable	Route Quality Class		
		High	Medium	Low
Stop-level	# Stops	140.60 (±33.31)	153.99 (±27.57)	151.25 (±31.65)
	# Stops w. Time Windows	9.54 (±7.31)	10.00 (±7.36)	3.12 (±2.87)
Package-level	# Packages	239.71 (±31.76)	237.19 (±30.23)	242.69 (±32.21)
	# Failed Delivery Attempts	2.06 (±3.21)	1.59 (±2.72)	1.70 (±2.81)
Route-level[a]	Route Duration (hrs)	3.30 (±0.82)	3.80 (±0.71)	3.81 (±0.89)
	Slack (min)	3.70 (±10.65)	3.78 (±11.64)	14.86 (±19.74)
	Delay (min)	3.13 (±12.88)	4.89 (±16.57)	9.05 (±20.63)

[a] Courier-performed routes

specificity further. `LA+A-2` serves as an example, where `-2` narrows down the location to a more specific part within the supra zone. The intra zone goes a step further and includes another integer from the zone ID; `LA+A-2.3`. The loci zone adds the final layer of detail, represented by a letter following the intra zone designation, such as `LA+A-2.3B`. Approximately 2% of the stops in the dataset lack any zone association.

- Lastly, the **route-level** data encompasses broader logistical details, including the originating depot, the capacity of the delivery truck, the date and time of dispatch, the sequence in which the stops were visited by the delivery driver, and the route's quality label.

3.2. Descriptive statistics

Table 1 compares the three route categories within the dataset. It highlights differences in route efficiency metrics, including route duration (only considers the transit time and excludes service times), slack (defined as the minutes arrived before a delivery time window begins), and delay (the minutes arrived after the delivery time window closes). Notably, high-quality routes demonstrate higher efficiency, having the shortest average route durations and minimal slack and delays. The gap in the average slack and delay values is larger between low- and medium-quality routes than between medium- and high-quality routes. Additionally, low-quality routes have fewer stops with assigned time windows and exhibit a lower number of failed delivery attempts than high-quality routes. For more descriptive statistics, refer to Merchán et al. (2022).

Fig. 1 shows the proportions of the quality classes by cities in the dataset. For each quality class, two scenarios may have occurred: (1) all delivery attempts were successful, or (2) at least one delivery attempt was unsuccessful. We refer to the routes falling under the former category as *Delivered* routes, and call the second category *Failed*. Amazon's unseen dataset for ranking the competitors only included *High+Delivered* routes (i.e., high-quality routes with all successful delivery handoffs) (Cook et al., 2022). It is important, therefore, to correctly sample from the dataset to ensure the representativeness of High+Delivered routes in the training and testing sets. Los Angeles, for instance, has the highest ratio of failed delivery attempts among all quality classes compared to the other cities. As seen in Fig. 1, the quality classes in different cities are imbalanced; specifically, low-quality routes are scarce (i.e., less than 2% of the entire dataset are labelled low-quality). This scarcity hindered the efficacy of a three-label classifier; therefore, we decided to merge the low and medium-quality routes and pivot to a binary classification system. Note that a classifier adept at distinguishing high-quality routes from the rest can still effectively guide a TSP solver towards favourable solutions.

3.3. Data splitting strategy

This study uses an 85/15 strategy for splitting the data into training and testing subsets. Throughout the splitting process, we use stratified random sampling considering three elements: (1) route quality class, (2) originating city, and (3) delivery status, to ensure that the instances are well represented in the training and testing subsets. As shown in Fig. 1, in some cities, there are not enough Low+Delivered or Low+Failed routes to divide between training and testing sets. We, therefore, dropped the delivery status stratification for low-quality routes and use five-fold cross-validation (i.e., we merge the medium- and low-quality routes into a single "not high-quality" class), with the above-mentioned stratification procedure, to evaluate the classifier's generalization ability by iterative training and validating the ML model on different combinations of folds (Dietterich, 1998).

4. Methodology

4.1. Framework overview

We propose a framework with two phases: offline and online, as illustrated in Fig. 2. In the offline phase, the framework trains an ML classifier on the historical delivery data using supervised learning. The classifier provides a probabilistic view of a feasible tour's quality in the field. We use the insights generated by the classifier to design a data-informed insertion heuristic, called the

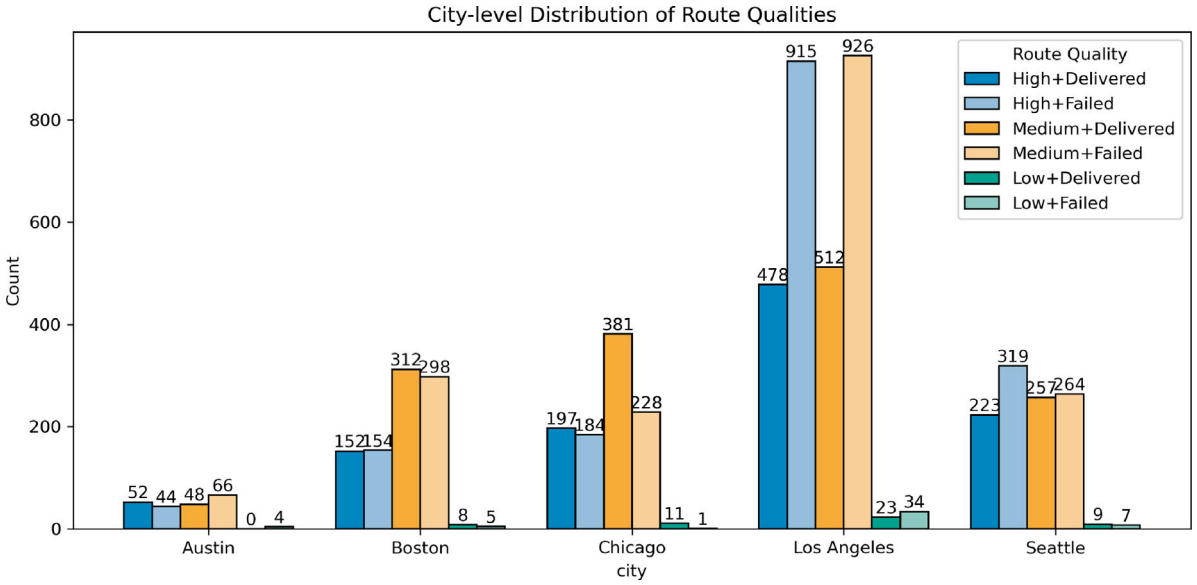


Fig. 1. Route quality distribution by city. The logistics planners at Amazon categorized the realized routes into one of three quality groups (i.e., high, medium, and low). Each route may or may not have unsuccessful delivery handoffs, for which the driver may or may not have retried the delivery within the same route. The delivery attempts in routes labelled 'Delivered' have all been successful, whereas 'Failed' represents routes with at least one failed delivery attempt.

DIIH, to generate high-quality solutions for future instances. In the online phase, the quasi-autoregressive heuristic dynamically inserts unvisited stops into a partial tour based on a cost function that contains additional variables to travel time. The cost function determines the most likely insertion position for a stop so that the prescribed tour is of high quality. The heuristic generates a pool of solutions (subject to a one-second runtime cap) and uses the classifier to prescribe the highest-quality solution to the driver. Sections 4.2, 4.3, and 4.4 present the notation adopted in this study, the ML classifier, and the structure of the DIIH, respectively.

4.2. Notation

Let $V = \{v_0, v_1, \ldots, v_n\}$ be the set of stops within an instance of the problem (v_0 represents the depot), where a set of packages are to be delivered at each stop $v_i \in V \backslash v_0$. Let $s \in S$ represent a feasible sequence in which the stops in V are visited, with S showing the solution space of all possible sequences. Solution s starts and ends at v_0 and visits $v_i \in V \backslash v_0$ exactly once. Extracting spatiotemporal patterns on the stop level is challenging since individual customer data might be too sparse. We reduce the data sparsity by partitioning the service area into zones. We use a zoning scheme that extracts more spatial information and enables easy access to the neighbours of each zone compared to Amazon's zoning scheme. To this end, we tessellate the service region into a hexagonal grid G^r using the `h3` geospatial indexing, where resolution r dictates the granularity of the tessellation (Uber, 2023). Each cell $g^r \in G^r$ represents a distinct *neighbourhood* (i.e., hierarchical spatial cluster) for a high-level view of the visit sequences and extracting patterns. We set $r \in [5, 10]$; a grid resolution of five (i.e., cell area of 253 km^2) is typically broad enough to include an entire route, and a resolution of 10 (i.e., cell area of $15,000\,\text{m}^2$) contains less than five stops within a route. For each instance, the stops V can be mapped to G^r to find the set of active cells $G^r_{active} \subset G^r$.

4.3. Machine learning classifier

4.3.1. Feature engineering

This study has engineered over 250 features (i.e., dependent variables), of which, the primary features are listed in Table 2. The rest of the features are derived from these primary ones through combination, discretization, and transformation. Each feature in the dataset is standardized by subtracting its mean and dividing it by its standard deviation in the training subset. The features in Table 2 are broadly categorized into two groups: instance-related and route-related. The former represent elements of the problem instance that are not influenced by the order of stops within a tour (e.g., the number of stops in the instance, the number of packages to be delivered, and the expected service time). Meanwhile, the route-related features depend on the stop orders (e.g., route duration and the degree of compliance with order time windows). The following discusses the key route-related features considered in this study:

Backtracking distance. Backtracking refers to the scenario where a driver, after making a delivery in a particular neighbourhood, needs to return to the same neighbourhood to make another delivery. The total distance covered during these back-and-forth

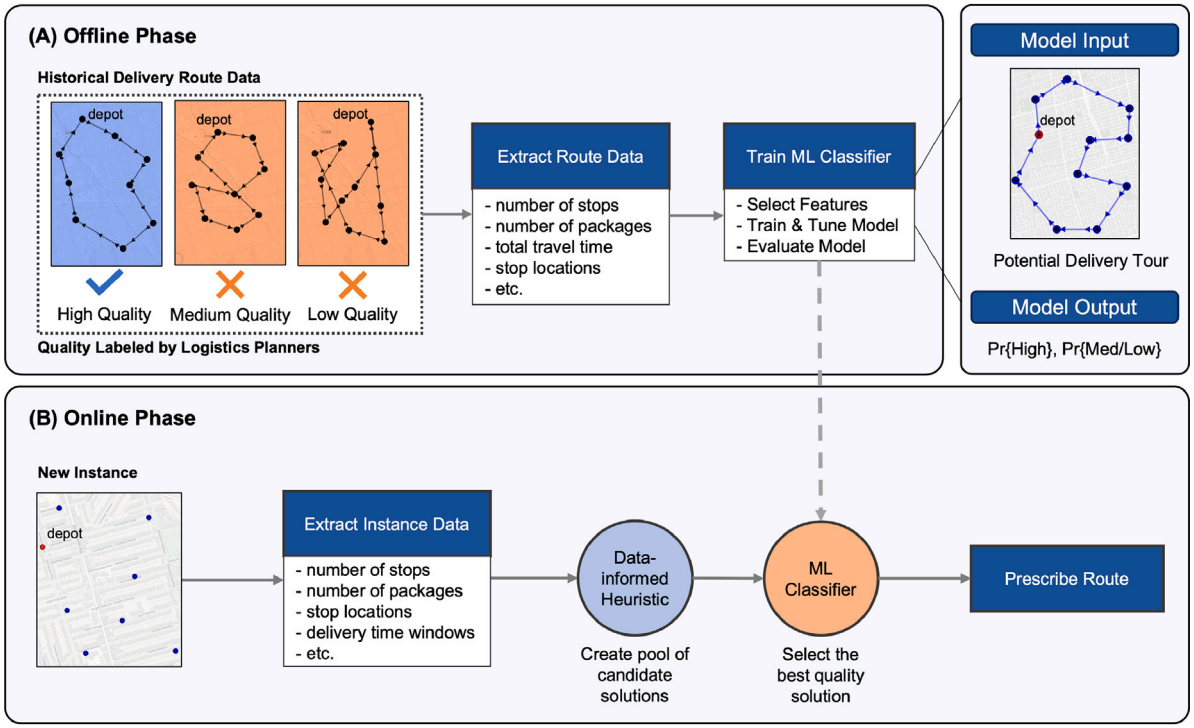


Fig. 2. Visual abstract of the proposed framework. VeRoViz (2023) was used for generating the sample route visuals.

Table 2
Pool of features.

Group	Feature Name	Description
Instance-related	numStops	Number of stops in the instance.
	numPackages	Total number of packages in the instance.
	totalServiceTime	Total expected service time for all stops.
	numActiveCells[a]	Number of active cells in the instance.
	numStopsTW	Number of stops with an assigned time window.
	dispatchTime	Time at which the driver is dispatched.
	avgTravelTimeDepot	Average travel time from each stop to the depot.
	vehicleCapacity	Capacity of the delivery vehicle.
	dayType	Weekday or weekend.
	city	The city where the delivery route is performed.
Route-related	routeDuration	The total duration of the delivery route, excluding service time.
	slackTime	Total minutes arrived earlier than the designated time window.
	delayTime	Total minutes arrived later than the designated time window.
	backtrackDistance[a]	Distance the driver had to backtrack during the route.
	bendingEnergy[a]	The sum of turn angles along the route.
	visitTimeQual[a,b]	The quality associated with routes' neighbourhood visit times.

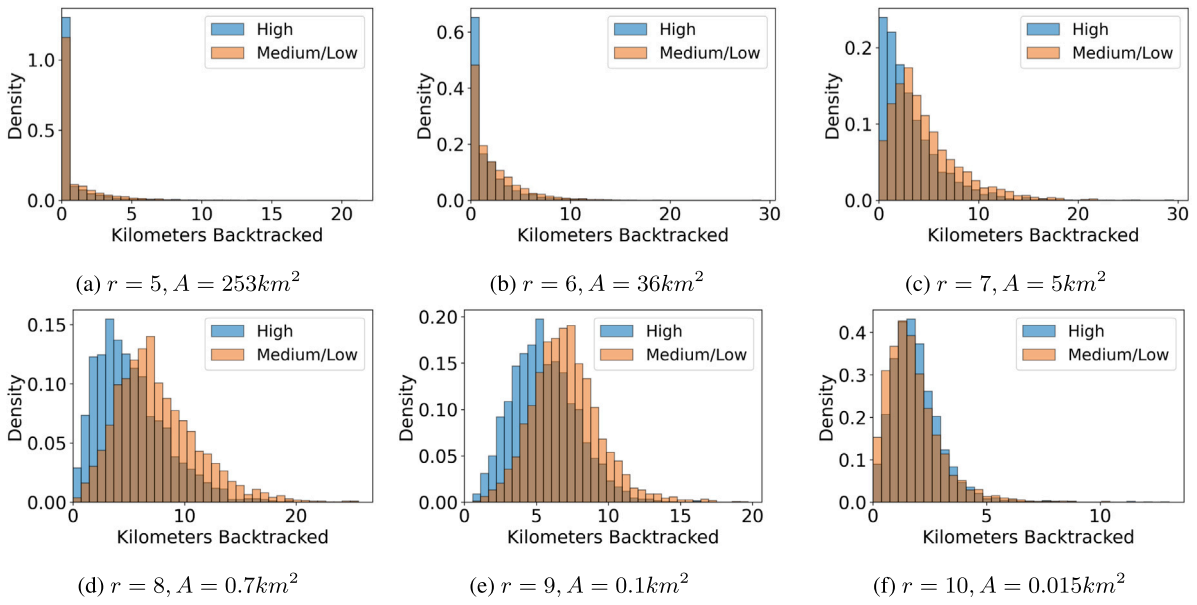
[a] Function of cell resolution,
[b] Function of time-of-day

movements is the backtracking distance (see Algorithm 1). Note that backtracking does not refer to the repetition of individual delivery stops or nodes. Such repetitions would violate the standard constraints of the TSP. We define backtracking at the neighbourhood or zone level, meaning that while each stop is visited exactly once, a driver may re-enter the same broader area or zone multiple times across the route.

Backtracks in a delivery route $s \in S$ are determined by mapping its stops onto the grid of cells, G^r, at resolution r, resulting in an active cell set $G^r_{active} \subset G^r$. The sequence of visits within G^r_{active} is inferred based on the known sequence of stops in s. When a backtrack is triggered, the distance between the origin and destination stop is taken as the backtracked distance. Fig. 3 suggests that high-quality routes in the training set have smaller backtracked distances, particularly in cell resolutions of eight and nine. Fig. 4 shows the variation in the backtracked distance based on the Amazon zoning scheme. The comparison of the two figures suggests that we extract more variation by using the tessellation scheme. We, nevertheless, consider spatial features based on both schemes in our pool of candidate variables.

Algorithm 1 Calculate Backtracked Distance

```
G^r_active ← ∅                                   ▷ Initialize active cells in the instance
BD ← 0                                           ▷ Initialize backtracked distance
prev_cell ← None                                 ▷ Initialize previous stop's cell
prev_stop ← None                                 ▷ Initialize previous stop
for stop in s do
    current_cell ← Cell(stop)                    ▷ Map the stop to a cell
    if prev_cell ≠ current_cell then
        if current_cell in G^r_active then
            distance ← geodesic_distance(current_stop, prev_stop)
            BD ← BD + distance
        else
            G^r_active ← G^r_active ∪ {current_cell}
        end if
    end if
    prev_cell ← current_cell
    prev_stop ← current_stop
end for
return BD
```



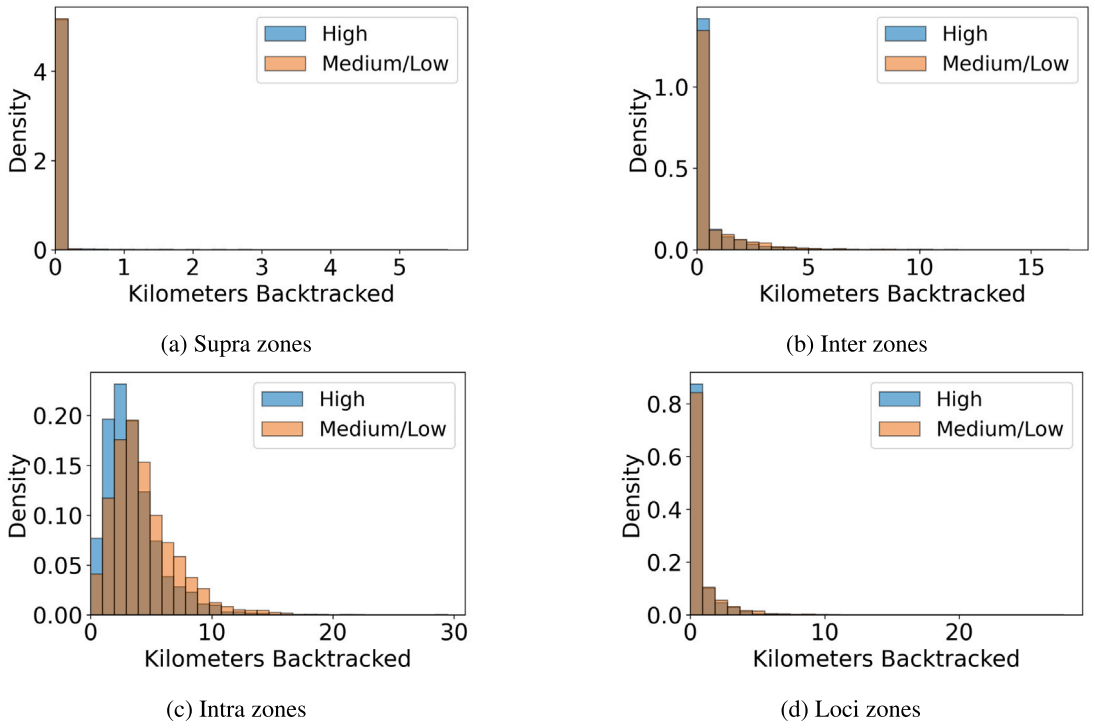
(a) $r = 5, A = 253km^2$ (b) $r = 6, A = 36km^2$ (c) $r = 7, A = 5km^2$

(d) $r = 8, A = 0.7km^2$ (e) $r = 9, A = 0.1km^2$ (f) $r = 10, A = 0.015km^2$

Fig. 3. Distance covered in backtracks distinguished by labelled route qualities across different cell resolutions. Cell resolutions and areas are denoted by **r** and **A**, respectively.

Bending energy. The routing literature suggests that *visually attractive* routes are generally better perceived by drivers and have a higher chance of conformance (Vickers et al., 2006; Rossit et al., 2019). While visual attractiveness is a subjective matter, the literature introduces *Intra-route Crossings* and *Bending Energy* as relevant metrics to quantify attractiveness in TSP solutions. Poot et al. (2002) defined *Intra-route Crossings* to count the number of times the edges in a TSP solution intersect one another. This study does not use the mentioned metric due to its time complexity (i.e. $\mathcal{O}(|V| \log |V|)$). *Bending Energy*, on the other hand, measures the sum of turn angles in a route and has a time complexity of $\mathcal{O}(|V|)$ (Gretton and Kilby, 2013). The inclusion of bending energy in the objective function penalizes sharp or frequent turns that result in zigzagging patterns, which can increase travel time, fuel consumption, and vehicle wear. Fig. 5 shows how bending energy may affect the quality of a route. Eq. (1) calculates the average of the smaller angles between arcs connecting stops $(i-2, i-1)$ and $(i-1, i)$, denoted by $\alpha_{(i-2,i-1,i)}$:

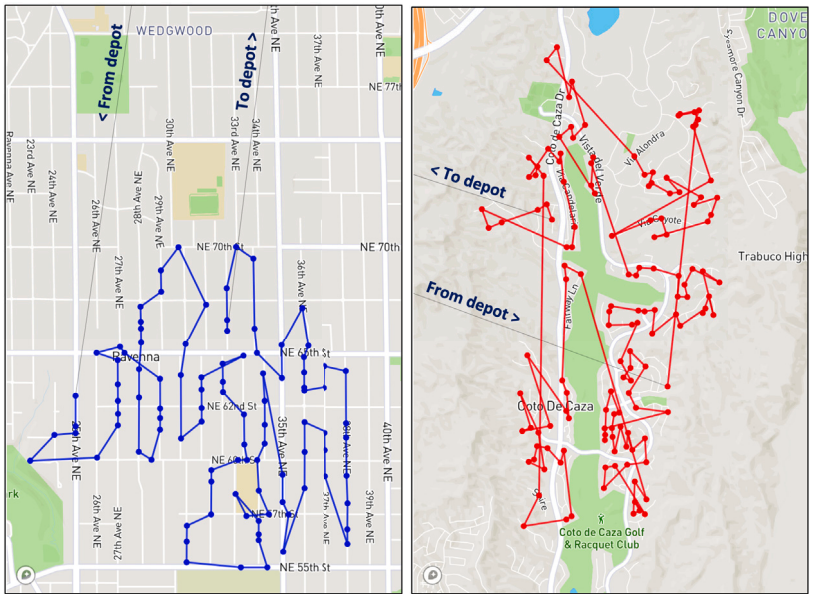
$$bendingEnergy = \sum_{i=2}^{|V|} \alpha_{(i-2,i-1,i)} \tag{1}$$



(a) Supra zones (b) Inter zones

(c) Intra zones (d) Loci zones

Fig. 4. Distance covered in backtracks distinguished by labelled route qualities across the Amazon geographical planning areas.



(a) High-quality route with high bending energy (b) Low-quality route with low bending energy

Fig. 5. Comparison of the bending energy in a high-quality and low-quality route.

Quality associated with the time of visit. We hypothesize that experienced drivers choose their visiting times to a neighbourhood based primarily on their accumulated knowledge of congestion patterns and recipient availability. The visitTimeQual metric aims to capture this latent information using a low-dimensional graph embedding and help quantify the quality of neighbourhood visit times of a solution $s \in S$, based on historical data. Assume that the quality of visiting a stop $v_i \in V$ within solution s at time t

is denoted by $q_{(v_i,t)}$. This study extracts two pieces of information from the Amazon dataset to estimate $q_{(v_i,t)}$ as $\hat{q}_{v_i,t}$: (1) the portion of historical instances with a visit to v_i at time t that are labelled high-quality, $P(\text{HQ}|v_i,t)$, and (2) the ratio of successful package delivery attempts for deliveries to v_i at time t, $P(\text{SD}|v_i,t)$. Eq. (2) uses the mentioned probabilities to calculate $\hat{q}_{v_i,t}$:

$$\hat{q}_{v_i,t} = P(\text{HQ}|v_i,t) \times P(\text{SD}|v_i,t) \tag{2}$$

where the first term, $P(\text{HQ}|v_i,t)$, serves as a proxy for determining how well a visit time t to stop v_i fits within the structure of historical high-quality routes. The second term, $P(\text{SD}|v_i,t)$, captures the likelihood of recipient availability at time t. Their product is a data-driven estimate of the suitability of visiting v_i at time t, both in isolation and as part of a complete route. This study assumes that $P(\text{HQ}|v_i,t)$ and $P(\text{SD}|v_i,t)$ are independent for simplicity. The recurrence of visiting a stop v_i at a specific time t is unlikely. This limits our ability to calculate meaningful values for $P(\text{HQ}|v_i,t)$ and $P(\text{SD}|v_i,t)$, which prevents an ML model from detecting useful patterns to learn from. To treat the sparsity, therefore, we map v_i to cell $g^r_{v_i} \in G^r_{active}$ at resolution r, and t to an interval of time u^h_t with bin size of h. Eqs. (3) and (4) calculate $P(\text{HQ}|v_i,t)$ and $P(\text{SD}|v_i,t)$, respectively:

$$P(\text{HQ}|v_i,t) = \frac{|W_{HQ}|}{|W|} \times \sqrt{\frac{|W|}{|W|+1}} \tag{3}$$

$$P(\text{SD}|v_i,t) = \frac{|R_{SD}|}{|R|} \times \sqrt{\frac{|R|}{|R|+1}} \tag{4}$$

where W shows the set of historical instances where $g^r_{v_i}$ was visited during u^h_t, and $W_{HQ} \subseteq W$ is the subset that is labelled high-quality. Similarly, R shows the set of historical delivery attempts to $g^r_{v_i}$ during u^h_t, and $R_{SD} \subseteq R$ is the subset that was successful. Using a simple ratio (e.g., $|R_{SD}|/|R|$) for determining the outcome of an event (e.g., the likelihood of a successful delivery handoff) is biased; we are generally more certain of the occurrence of an event if the event has been observed more frequently in the historical dataset, or in other words, the event has a higher *historical coverage*.

As an example, assume that the historical data suggests that out of the 200 deliveries attempted between 10–11 AM to neighbourhood A, 180 deliveries were successful. For neighbourhood B, however, only 20 have been attempted and 18 of which were successful. While both these scenarios have a $|R_{SD}|/|R| = 90\%$, we should be more certain of the success rate of the neighbourhood with more delivery attempts. We correct for this bias by penalizing the number of items in sets W and R, as shown in Eqs. (3) and (4). This study chose the correction function structure of $f(x) = \sqrt{x/(x+1)}$ due to the following reasons: (1) $f(x)$ does not penalize a probability with relatively high coverage, since $\lim_{x\to+\infty} f(x) = 1$, (2) the square root in $f(x)$ controls the penalization magnitude and prevents heavy penalization, and (3) $f(x)$ assumes that information with lower coverage is overestimated and acts conservatively to penalize them. Using Eq. (4) we may calculate the success rates of the before-mentioned example as 0.897 and 0.878, respectively.

We calculate the visitTimeQual of a tour using Eq. (5), which aggregates the quality values $\hat{q}_{v_i,t}$ across all stops with historical data. Some stops may lack historical coverage (i.e., their $\hat{q}_{v_i,t}$ value is unavailable); in such situations, we do not include those stops in the summation for visitTimeQual. While the visitTimeQual sum is helpful it still may be flawed; assume two routes, A and B, where out of the 10 stops in A, three have historical coverage, and their $\hat{q}_{v_i,t}$ values are equal to 0.5. Route B, on the other hand, has 20 stops in total with full historical coverage and their $\hat{q}_{v_i,t}$ values are all equal to 0.9. The visitTimeQual for A and B is similar and close to -0.91, while their individual stop visit time qualities are quite different. To address this discrepancy and estimate a more comparable measure of route quality, we divide the visitTimeQual by the ratio of stops on the route that have historical coverage. This ratio extrapolates the quality for stops without direct historical data. Both the visitTimeQual sum and the extrapolated route quality (referred to as extVisitTimeQual) are incorporated as candidate features in our classifier.

$$\text{visitTimeQual} = \log\left(\prod_{v_i \in V} \hat{q}_{v_i,t}\right) = \sum_{v_i \in V} \log(\hat{q}_{v_i,t}) \tag{5}$$

In order to have a faster inference in the routing heuristic, we calculate the $P(\text{HQ}|v_i,t)$ and $P(\text{SD}|v_i,t)$ for all cells in the historical instances offline and store them in a Python Dictionary (Python, 2024) to allow for efficient lookups. The average retrieval time from a dictionary is constant and is not affected by the dictionary's size. The dictionary uses the historically realized cells as keys and creates a list of $\hat{q}_{v_i,t}$ values for each interval of time during the day. Several considerations are in place for creating the dictionary: (1) The target variable (i.e., route quality class) is used in the process of calculating $P(\text{HQ}|v_i,t)$, and would leak data during training. We, therefore, create a dedicated dictionary for each instance in the training set without the instance. Note that during testing, the ML model has already been trained. Hence, one dictionary is created based on the instances in the training set. (2) If all visits to a cell during a time interval are labelled medium/low quality, then $P(\text{HQ}|v_i,t) = 0$. In these cases, we use a small probability value (i.e., 0.01) instead of zero to avoid fully penalizing the $\hat{q}_{v_i,t}$ value and losing information about recipient availability. (3) Once the dictionary is created, we interpolate the $\hat{q}_{v_i,t}$ values for unvisited cells using their one-hop neighbours (i.e., neighbours within the immediate vicinity of a neighbourhood). First, we find all neighbours to the present cells in the dictionary and then estimate their $\hat{q}_{v_i,t}$ value for each time bin by naively averaging the $\hat{q}_{v_i,t}$ values of their neighbours.

4.3.2. Selecting features for the classifier

This study drops the highly correlated features using the Spearman correlation test to prevent multicollinearity (Schober and Schwarte, 2018). The Spearman correlation test examines the monotonic relationships between ordinal and continuous variables. After removing these correlated features, we fed the remaining non-correlated features into our model for training. To prevent overfitting while training the feature parameters, we used a regularization term and tuned the regularization weight (see Section 4.3.4). We conducted a random search to find the value leading to the highest model accuracy. For each candidate regularization weight, we used stratified five-fold cross-validation to evaluate the model's performance and generalizability (see Section 3.3). In each iteration of the cross-validation, we trained the model on four folds and validated it on the remaining one. We repeated this process for all five folds and calculated the average model accuracy across the validation sets for each regularization weight. By comparing the average validation accuracies, we identified the regularization weight that achieved the highest performance. Appendix B examines the impact of different features on the model's performance.

4.3.3. Model specification

We train and compare four candidate classifiers: Logistic Regression (LR), Multi-layer Perceptron (MLP), Random Forest (RF), and a custom Graph Neural Network with weighted messages (GNN-wm) (Breiman, 2001; Peng et al., 2002; White and Rosenblatt, 1963; Hamilton, 2020). These models differ primarily in their data processing and operational mechanisms. LR, RF, and MLP typically use tabular data (e.g., type of data explained in Section 4.3.1), requiring the pre-aggregation or flattening of graph data. However, this process can result in the loss of some relational and structural information that is inherent in the graph, which may not be fully captured during preprocessing. In contrast, GNNs are designed to directly use a graph with node-, edge-, or graph-level features as input, potentially capturing more complex patterns and interactions. GNNs often require more data to effectively leverage these capabilities, similar to other complex machine learning models. Past data-driven optimization literature has also opted for simpler models in the presence of more complex ones (Liu et al., 2021; Özarık et al., 2023).

Fig. 6 shows the GNN-wm's three-layer training pipeline: input, embedding learning, and prediction layers. In the input layer, the network receives a graph $\mathcal{G} = (V, E)$ representing a feasible TSP tour conforming to a visiting sequence $s \in S$. This graph $\mathcal{G}$ contains features at three levels: node-level, edge-level, and graph-level. For each node $v_i \in V$, the features are denoted as $x_i \in \mathbb{R}^{d_n}$, where d_n represents the dimensionality of the node features. These features include the coordinates, arrival time (inferred from the visiting sequence), and a binary variable indicating the presence of a time window. For edges $e_{ij} \in E$ connecting two adjacent nodes v_i and v_j, the edge features are denoted as $x_{ij} \in \mathbb{R}^{d_e}$, where d_e is the dimensionality of the edge features, capturing the travel time between the nodes (i.e., $d_e = 1$). At the graph level, the feature is the bending energy of the tour, denoted as $x_g \in \mathbb{R}^{d_g}$, where d_g represents the dimensionality of the graph features, and this is calculated using Eq. (1) (i.e., $d_g = 1$).

The input layer in the GNN-wm maps these features to their initial hidden embeddings with dimensionality d. For nodes, the initial embeddings are represented as $h_i^{(0)} \in \mathbb{R}^d$ for node v_i; for edges, as $e_{ij}^{(0)} \in \mathbb{R}^1$ for the edge connecting v_i and v_j; and for the graph, as $g^{(0)} \in \mathbb{R}^d$. This mapping is achieved through a linear transformation: $h_i^{(0)} = W_n x_i + b_n, \quad e_{ij}^{(0)} = W_e x_{ij} + b_e, \quad g^{(0)} = W_g x_g + b_g$, where $W_n \in \mathbb{R}^{d \times d_n}$, $W_e \in \mathbb{R}^{1 \times d_e}$, and $W_g \in \mathbb{R}^{d \times d_g}$ are weight matrices, and b_n, b_e, and b_g are biases for nodes, edges, and the graph, respectively. The edge feature hidden embedding is represented with a weight matrix $W_e \in \mathbb{R}^{1 \times d_e}$, resulting in a scalar value since the edge embedding is only used as a weight for message passing. This approach reduces the number of learnable parameters, though a more complex representation for the edge embeddings can be used if needed.

The embedding learning layer generates more complex representations of the nodes and the graph features. We refine the graph- and node-level embeddings using an MLP and multiple layers of graph convolutions, respectively. The convolutional layer updates node embeddings by aggregating information (i.e., messages) from neighbouring nodes. These messages are weighted by the edge embeddings that connect the nodes, as shown in Eq. (6):

$$h_i^{(l+1)} = \text{UPDATE}\left(h_i^{(l)}, \text{AGGREGATE}\left(\left\{\frac{h_j^{(l)}}{e_{ij}^{(0)}} \mid j \in \mathcal{N}(i)\right\}\right)\right) \tag{6}$$

where, $h_i^{(l)}$ represents the embedding of node i at layer l, and $h_j^{(l)}/e_{ij}^{(0)}$ denotes the *message* from node j to node i. The term $\mathcal{N}(i)$ refers to the set of neighbours of node i. Messages have less impact if they come from nodes that are farther away, as indicated by the edge embedding $e_{ij}^{(0)}$. The functions AGGREGATE and UPDATE determine how messages are combined and how the node embedding is updated, respectively. The classical GNN literature has proposed and used various functions for these steps; we select the best-performing set of functions during hyperparameter tuning, as explained in Section 4.3.4 (Hamilton et al., 2017; Veličković et al., 2017; Kipf and Welling, 2017). We use two layers of convolution to avoid over-smoothing the node representations.

In the prediction layer, the node embeddings are pooled to form a graph-level embedding. They are then concatenated with the graph embedding and fed into another MLP, followed by a linear transformation, to predict the quality class of the TSP route. The parameters of the GNN-wm are optimized using a cross-entropy loss function: $L = -\sum_c y_c \log(\hat{y}_c)$, where y_c is the true label and $\hat{y}_c$ is the predicted probability for class c.

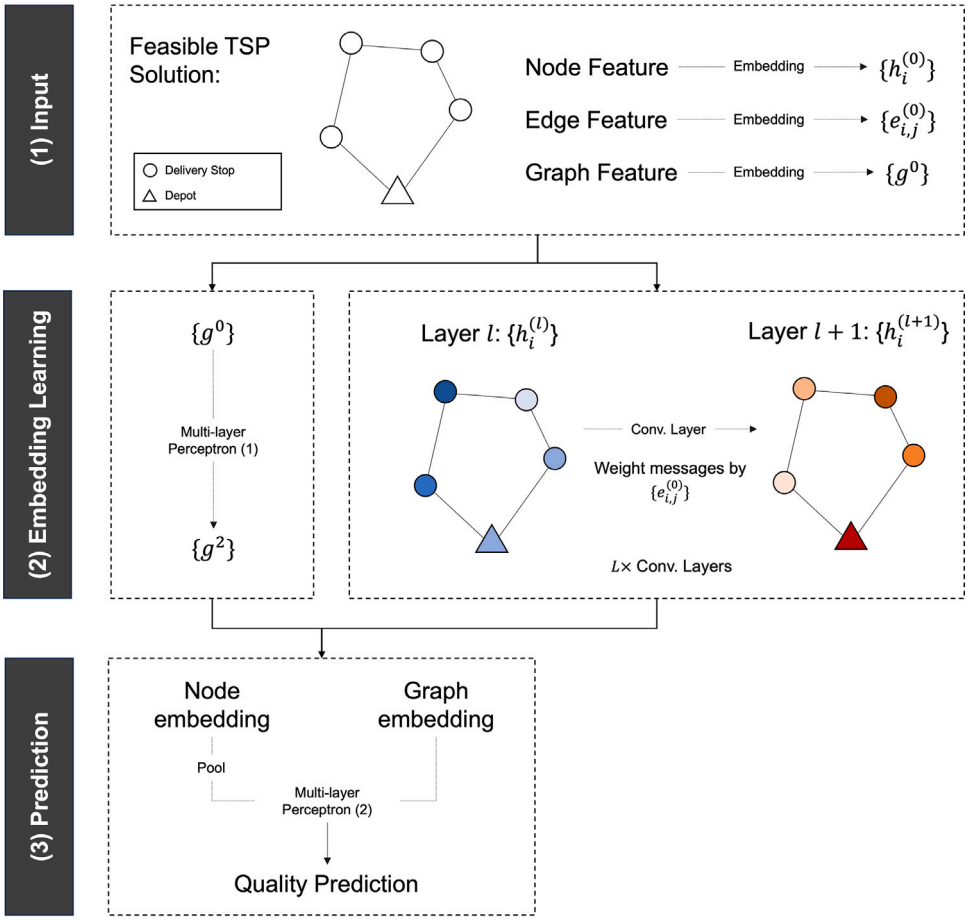


Fig. 6. The high-level architecture of the proposed Graph Neural Network with weighted message-passing.

4.3.4. Hyperparameter tuning

We perform a random search across each model's hyperparameter grid to find the combination that results in the highest classification accuracy. After tuning the parameters, we tune the decision-making threshold to ensure maximum accuracy. The parameters of the models are explained in the following:

Logistic Regression. We consider (1) whether to include an L2 regularization penalty. The L2 regularization, adds a penalty equal to the square of the magnitude of coefficients. This helps to prevent overfitting by discouraging overly complex models. (2) If regularization is applied, then we tune its strength. A stronger regularization leads to more shrinkage, i.e., the coefficients are pushed towards zero, which is useful in preventing overfitting. Weaker regularization values allow the learning algorithm to prioritize fitting the training set, which can be useful if the model is underfitting. (3) Whether to consider a constant value (i.e., intercept) for the model. (4) Whether to adjust class weights to counteract imbalances, with higher penalties applied to errors in the underrepresented class.

Random Forest. (1) The number of estimators; controls the number of trees in the forest; more trees generally improve the model's accuracy and robustness but at the cost of increased computational demand. (2) The depth of each tree; which helps to prevent overfitting by curbing the complexity of the decisions trees can make. (3) The minimum number of samples and leaves; these parameters ensure that each node and leaf in the trees have a minimum number of samples, which aids in generalizing the model by preventing the learning of excessively fine-grained patterns that do not generalize well to new data. (4) The class weight parameter; we tune the parameter to address the class imbalance by modifying the weight that misclassified instances of the minority class contribute to the model's loss.

Multi-layer Perceptron. (1) The number of hidden layers and activation functions; which influence the model's complexity and capacity to capture complex patterns in the data. (2) The solver for weight optimization; which typically affects the speed and quality of training by adjusting how quickly and effectively the model finds optimal weights. (3) The regularization term, that helps prevent overfitting by penalizing large weights. (4) Learning rate adjustment to dictate how learning progresses, potentially improving the model's ability to find and converge to the global minimum. Finally, we tune (5) whether to use early stoppage to halt training when validation performance degrades.

Graph Neural Network with Weighted Messages. We consider (1) the type of convolution layer and its specific configuration. We focus on three commonly used models: Graph Convolutional Networks (GCN), Graph Sample and Aggregated (GraphSAGE), and Graph Attention Networks (GAT) (Hamilton et al., 2017; Kipf and Welling, 2017; Veličković et al., 2017). (2) The learning rate and weight decay; the learning rate determines the step size during optimization, while weight decay serves as a regularization term to prevent overfitting. (3) The number of hidden dimensions, which controls the size of the feature representations and influences the model's capacity to capture complex patterns. We fix the network depth (depth of MLPs and the convolutional layers) to two layers to concentrate tuning efforts on other aspects of the pipeline. (4) The pooling function, which aggregates node features into a graph-level representation (i.e., sum, mean, max).

4.4. Data-informed insertion heuristic

4.4.1. Heuristic outline

Insertion heuristics create solutions by greedily inserting stops into a partial solution; they choose *which* stop to insert and *where* to insert the stop (Campbell and Savelsbergh, 2004). This study uses a random insertion heuristic, where stops are randomly selected for insertion, and a data-informed cost function is created to determine the best insertion position. Algorithm 2 shows the outline of the DIIH developed in this study. The algorithm starts by realizing the set of stops to cover and creating an initial solution with two depot visits. In lines 4–12, the heuristic randomly selects an unvisited stop and uses a cost function to determine the best insertion position for it. The cost function's structure and time complexity are discussed in Section 4.4.2. Once the best insertion position is determined, the algorithm updates the partial tour and set of unvisited stops in lines 13 and 14, respectively. When all stops are visited in the tour, a feasible solution is returned. Note that in our proposed framework we use the DIIH to generate a pool of solutions subject to a one-second cap to potentially escape from local optima (see Section 4.1).

Algorithm 2 Data-informed Insertion Heuristic

1: $V \leftarrow \{v_0, v_1, \dots, v_n\}$ ▷ Set of stops including depot v_0
2: $s \leftarrow [v_0, v_0]$ ▷ Initialize a solution with depot as start and end
3: $N \leftarrow V \setminus v_0$ ▷ Initialize the set of unvisited stops
4: **while** $N \neq \emptyset$ **do** ▷ While there are stops not yet visited
5: $random_stop \leftarrow$ RANDOM(N) ▷ Select a random stop from unvisited stops
6: $pos^* \leftarrow$ null, $\Delta^* \leftarrow \infty$ ▷ Initialize best insertion variables for the random stop
7: **for** $i \in \{1, \dots, |s| - 1\}$ **do** ▷ Use data-informed cost function to determine the insertion position
8: $\Delta \leftarrow$ Cost($s \cup \{random_stop\}$ at position i) − Cost(s)
9: **if** $\Delta < \Delta^*$ **then** ▷ If cost improvement, update best insertion
10: $pos^* \leftarrow i, \Delta^* \leftarrow \Delta$
11: **end if**
12: **end for**
13: $s.insert(pos^*, random_stop)$ ▷ Insert the random stop at the best position
14: $N \leftarrow N \setminus \{random_stop\}$ ▷ Remove the inserted stop from unvisited set
15: **end while**
16: **return** s ▷ Return the constructed solution

4.4.2. Cost function

The cost function in this study is built around the insights derived by the ML classifier. It evaluates the cost of a candidate insertion position based on the change in travel time Δ_{TT}, backtracked distance Δ_{BD}, bending energy Δ_{BE}, and the quality associated with visit times Δ_{VTQ}. The incremental improvement in the quality of a partial solution is quantified by the equation $\Delta = \Sigma_{f \in F} \beta_f \Delta_f$, where F shows the pool of features. For a feature $f \in F$, the value of Δ_f is parameterized by β_f. In this study, we determine the parameters β_f using the ML classifier. The following explains how the Δ_f values are calculated within the cost function.

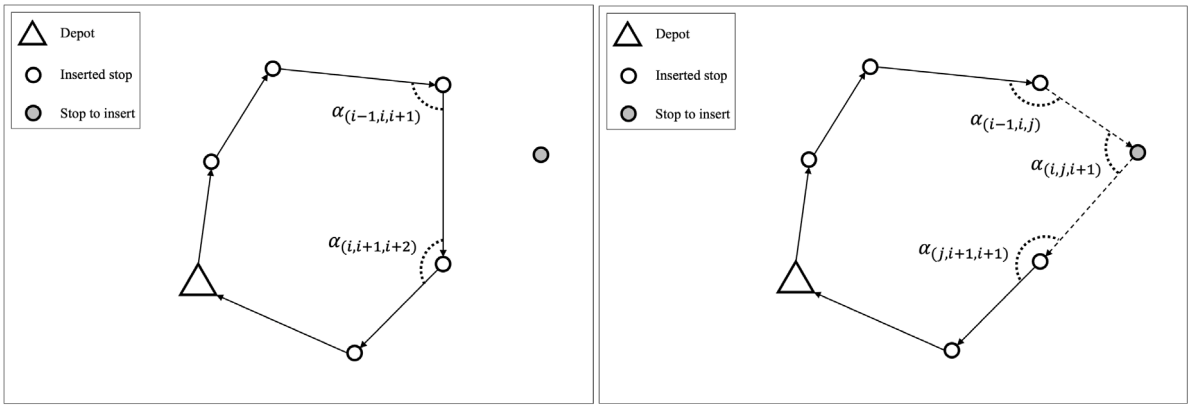
Travel time. The induced change in travel time Δ_{TT} as a result of inserting a stop at position j, between positions i and $i+1$ can be handled within $\mathcal{O}(1)$; $\Delta_{TT} = T_{i,j} + T_{j,i+1} - T_{i,i+1}$.

Backtracked distance. We do not calculate the backtracked distance directly within the cost function as it requires $\mathcal{O}(|V|)$ calculations in the worst scenario. Instead, we penalize backtracks by increasing the travel time between stops that are not within the same cell (see Algorithm 3). The algorithm increases the travel times by adding a large value M. This modification encourages the algorithm to sequence stops within the same cell consecutively, thereby reducing redundant re-entries across cells. Note that the process of modifying the travel times is done during preprocessing and is not performed in the DIIH's iterative process.

Bending energy. Fig. 7 shows a sample instance with six stops. In Fig. 7(a), a stop is to be inserted at position j between positions i and $i+1$. Upon insertion, the turning angles adjust and the value of the change in the bending energy Δ_{BE} can be calculated using Eq. (7):

$$\Delta_{BE} = [\alpha_{(i-1,i,j)} + \alpha_{(i,j,i+1)} + \alpha_{(j,i+1,i+2)}] - [\alpha_{(i-1,i,i+1)} + \alpha_{(i,i+1,i+2)}] \tag{7}$$

where n shows the number of stops in the partial route before insertion. Δ_{BE} is simply equal to the difference between the summation of the angles shown in Figs. 7(a) and 7(b). Eq. (7) can be calculated within $\mathcal{O}(1)$.



(a) Before stop insertion (b) After stop insertion

Fig. 7. Demonstration of the changing angles in a tour after inserting a stop.

Quality associated with visit times. The Amazon competition assumes that the driver dispatch time is known a priori. Using the dispatch times, we can recursively calculate the arrival time to a stop at position i using Eq. (8); where AT shows the arrival time to a stop, and ST is the service time. Eq. (8) is not an efficient approach for updating the stop arrival times following a potential insertion of an unvisited stop. For a stop inserted at position j, between positions i and $i+1$, the arrival times of AT_j and AT_{i+1} can be calculated using Eq. (8) and the stops from position $i+1$ onwards should be updated using a *time bump* equal to the difference of AT_{i+1} before and after the insertion. The process of updating arrival times will require $\mathcal{O}(|V|)$ calculations at worst. Once the arrival times are updated, the extrapolated visit time quality metric may be calculated as explained in Section 4.3.1, and set as the value of Δ_{VTQ}.

$$AT_i = AT_{i-1} + T_{i-1,i} + ST_{i-1} \tag{8}$$

Algorithm 3 Penalize Backtracks

$V \leftarrow \{v_1, \dots, v_n\}$ ▷ Set of stops excluding depot v_0
$\mathbf{T}^{modified} \leftarrow \mathbf{T}$ ▷ Initialize the modified travel time matrix
$M \leftarrow 0$ ▷ Initialize the penalty multiplier
for $i \in V$ **do** ▷ Find the maximum travel time
  for $j \in V$ **do**
    $M \leftarrow \max(M, T_{i,j})$
  end for
end for
$M \leftarrow M \times \frac{\beta_{BD}}{\beta_{TT}}$ ▷ Modify the value of M based on the ratio of parameters in the cost function
for $i \in V$ **do**
  for $j \in V \setminus i$ **do**
    if $Cell(i) \neq Cell(j)$ **then** ▷ Cell is defined by the granularity set by user
      $T^{modified}_{i,j} \leftarrow T_{i,j} + M$
    end if
  end for
end for
return $\mathbf{T}^{modified}$ ▷ Return the modified travel time matrix

5. Experimental setup

5.1. Baseline

Machine Learning Classifier. This study trains and compares several classifiers: logistic regression, random forest, a multi-layer perceptron, and a graph neural network.

Solution Generation Framework. The complete framework's performance is benchmarked to a solution with near-optimal total travel time and the methodology implemented by the winner of the Amazon competition, the work of Cook et al. (2022). We generate the near-optimal solution using `OR-tool`'s Guided Local Search (GLS) capped to a one-second runtime (Perron and Furnon, 2023). Our study was implemented using a Macbook Air with an eight-core Apple M2 chip and 16 GB of RAM.

5.2. Evaluation metrics

Machine Learning Classifier. The classifier's performance is measured using standard binary classification metrics applied during the cross-validation and testing phases. Accuracy gauges the model's overall prediction correctness, Precision assesses its positive predictive value, Recall measures its ability to find the true positives, and the Negative Predictive Value (NPV) indicates the proportion of correctly identified negatives among all predicted negative outcomes. The F1 score is employed to address the class imbalance, providing a balanced metric that considers both precision and recall. Further, the analysis uses the Receiver Operating Characteristic (ROC) curve, which plots the true positive rate against the false positive rate. The Area Under the ROC Curve (AUC) complements this by providing an aggregate performance measure across all possible classification thresholds, with values approaching 1 indicating a highly effective predictor.

Solution Generation Framework. The framework's effectiveness is evaluated based on the percentage of solutions that meet the criteria for high quality after applying the framework. Additionally, we use Amazon's disparity metric to evaluate how well the framework replicates the routes taken by drivers in the High+Delivered instances (MIT CAVE lab, 2021). As Amazon's testing subset does not include the ground truth routes, we evaluate our framework and benchmarks using the testing subset created in this study. The disparity score between sequences A and B is denoted by $R(A, B)$ and calculated using Eq. (9):

$$R(A, B) = \frac{SD(A, B) \times ERP_{norm}(A, B)}{ERP_e(A, B)} \tag{9}$$

where $SD(A, B)$, $ERP_{norm}(A, B)$, and $ERP_e(A, B)$ show the Sequence Deviation, Edit Distance with Real Penalty, and Edit Distance between sequences A and B. A score of $R(A, B) = 0$ denotes perfect similarity between the tested sequences. The Sequence Deviation measures how the order of stops in one sequence varies from another using Eq. (10):

$$SD(A, B) = \frac{2}{|V|(|V| - 1)} \sum_{i=2}^{|V|} (|a_{[i]} - a_{[i-1]}| - 1) \tag{10}$$

where $a_{[i]}$ denotes the index of the ith element of sequence B in sequence A. The Edit Distance, $ERP_e(A, B)$, determines the minimum number of changes (i.e., insertions, deletions, substitutions) required to make two sequences identical. $ERP_{norm}(A, B)$ calculates the Edit Distance with Real Penalty (ERP) between two routes—A and B. The metric determines the cost (based on the normalized travel time matrix) and the number of edits required to transform the A into the B. By employing a normalized cost matrix, the $ERP_{norm}(A, B)$ is a more contextually relevant measure of route similarity. The metric explores three possible changes: matching the current elements of both sequences, inserting a gap in the *observed* route, or inserting a gap in the *prescribed* route. Each recursive call of the metric assesses these options, adding up the cost from a given cost matrix if elements differ.

6. Results

6.1. Classifier performance and selection

As seen in Table 3, all models outperform naive models (e.g., random, all-ones, all-zeros), evidenced by higher accuracy during cross-validation. The performance of all models across cross-validation folds is stable, with low standard deviation values for all metrics. The average cross-validation scores match the test set results across all metrics, suggesting strong generalizability to unseen data. The GNN-wm has a slightly lower accuracy score than the LR, RF, and MLP, which may be attributed to the limited amount of training data. More data could potentially improve its performance. We chose the LR as the primary classifier to create the DIIH for the following reasons: (1) The LR has the best training performance across all evaluation metrics except precision, (2) the LR offers statistical information about the features' significance and role in determining the target variable, and (3) the LR is particularly well-suited for low-latency applications due to its simplicity and efficiency compared to more complex models. The hyperparameters of the trained models are reported in Appendix A.

Table 4 summarizes the modelling results of the LR classifier. The results indicate that both instance-related and route-related features are statistically significant in determining a route's quality. In Appendix B, a complementary permutation-based analysis of the feature importance is provided. The following explains the contribution of the features based on their statistical significance mentioned in Table 4 and comments about the potential insights that may be drawn:

Instance-related features. The instance-related features (i.e., numStops, numPackages, and numActiveCells) show that the structure of medium/low-quality routes is statistically different from that of instances with a courier-performed high-quality route. The LR suggests that routes with a larger number of stops, packages, or active cells have higher quality (i.e., their coefficients are positive). These features are potentially acting as proxies for route productivity. For instance, while controlling for all variables, routes with more packages are more productive and, thus, have higher quality.

Route duration. As expected, the time spent driving for a delivery route is a significant feature. Routes with higher driving times are perhaps less efficient. Note that this feature does not make the highest contribution to determining route quality.

Backtracked distance. The results indicate that the backtracks at two different cell resolutions (i.e. eight and nine) are significant. Backtracks at both resolutions have a negative impact on the route's quality, but they have different levels of impact. We tested reduced models (i.e., models with backtracks for a single cell resolution), and the results showed that the two resolutions provide uncorrelated information to the model. The two features are less than 60% correlated based on the Spearman and Pearson coefficients of correlation.

Table 3
Classifier performance during cross-validation and testing. For each metric, the best-performing model during training is shown in bold.

Model	Dataset	Accuracy	Precision	Recall	F1-score	NPV[a]	AUC[b]
Logistic Regression	CV Mean[c]	**0.704**	0.691	**0.604**	**0.645**	**0.712**	**0.758**
	CV Std dev[d]	0.009	0.010	0.025	0.015	0.011	0.015
	Test set	0.700	0.700	0.570	0.628	0.700	0.750
Random Forest	CV Mean	0.691	0.675	0.589	0.628	0.701	0.749
	CV Std dev	0.012	0.023	0.035	0.018	0.013	0.009
	Test set	0.686	0.671	0.575	0.619	0.695	0.748
Multi-layer Perceptron	CV Mean	0.700	**0.701**	0.567	0.627	0.699	0.757
	CV Std dev	0.005	0.013	0.019	0.009	0.006	0.014
	Test set	0.700	0.712	0.545	0.618	0.694	0.749
GNN witth weighted messages	CV Mean	0.688	0.672	0.585	0.625	0.699	0.747
	CV Std dev	0.007	0.009	0.022	0.014	0.009	0.014
	Test set	0.676	0.652	0.580	0.614	0.691	0.730
Random Model (weighted)	Test set	0.482	0.419	0.430	0.425	0.534	0.476
All ones	Test set	0.445	0.445	1.000	0.616	–	0.500
All zeros	Test set	0.555	0.000	0.000	0.000	0.555	0.500

[a] Negative predictive value,
[b] Area under curve,
[c] Cross-validation,
[d] Standard deviation

Table 4
Modelling results of the logistic regression model. Negative and positive coefficients decrease and increase the likelihood of having a high-quality route, respectively.

Category	Feature	Coefficient	Std. Err.	Z-value	*P*-value
	Intercept	−0.340	0.061	−5.577	<0.001
Instance-related	numStops	0.489	0.077	6.321	<0.001
	numPackages	0.093	0.047	1.990	0.047
	numActiveZones (r=7)[a]	0.316	0.059	5.343	<0.001
Route-related	routeDuration	−0.200	0.055	−3.618	<0.001
	backtrackDistance (r=8)	−0.205	0.061	−3.348	0.001
	backtrackDistance (r=9)	−0.423	0.051	−8.223	<0.001
	bendingEnergy (r=9)	−0.412	0.086	−4.772	<0.001
	visitTimeQual (r=7, h=2)[b]	0.278	0.060	4.622	<0.001
	visitTimeQualHQ (r=7, h=0.5) > −85	0.788	0.095	8.257	<0.001
	visitTimeQualMLQ (r=7, h=0.5) < −259	−0.272	0.105	−2.593	0.010

[a] r = Cell Resolution,
[b] h = Time bin length in hours

Bending energy. The bending energy across cell resolution nine was significant. The metric determines the sum of turning angles between the centroid of active cells of resolution nine in a route. A higher bending energy indicates that less turning is needed for a delivery route.

Quality of neighbourhood visit times: The LR shows statistical evidence that high-quality routes visit neighbourhoods at different times than when the medium/low-quality routes do. Routes with more similar neighbourhood visit times to historical high-quality routes have higher quality. Note that both the visitTimeQual sum and the extVisitTimeQual emerged as significant features in the model. The time bin size h is different between the two sets of features: the extrapolated features have a finer bin size, while the sum has a larger. This difference may be attributed to the fact that the extrapolated is more robust against the sparsity of data than the sum.

6.2. Framework's performance

The results indicate that our framework outperforms the benchmark by generating a higher proportion of high-quality solutions within the testing subset. As depicted in Fig. 8(a), the classifier's quality assessment of the solutions from each method reveals that our heuristic successfully generates high-quality solutions for 98.8% of the instances. This shows the success of the designed heuristic in capitalizing on insights from the classifier. It is important to note, however, that the classifier, while informative, is not infallible and possesses inherent limitations in its predictive power. To adjust for the classifier's predictive performance, we correct the proportions of expected high-quality versus medium/low-quality routes based on the classifier's precision score of 0.691 and an NPV of 0.712, as reported in Table 3. This correction implies that, on average, 69.1% of routes classified as high quality

Table 5
Performance of methods in terms of Amazon's disparity score.

Algorithm	Mean	Std Dev	Min	First Quart.	Median	Third Quart.	Max
Guided Local Search	0.091	0.050	0.010	0.055	0.076	0.105	0.330
Cook et al. (2022)	0.019	0.027	0.001	0.003	0.007	0.026	0.146
Our Framework	0.119	0.069	0.015	0.071	0.105	0.155	0.404

and $100 - 71.2 = 28.8\%$ of the routes predicted as medium/low quality are indeed high quality. Fig. 8(b) illustrates the adjusted distributions of route qualities, taking into account the classifier's precision and NPV metrics.

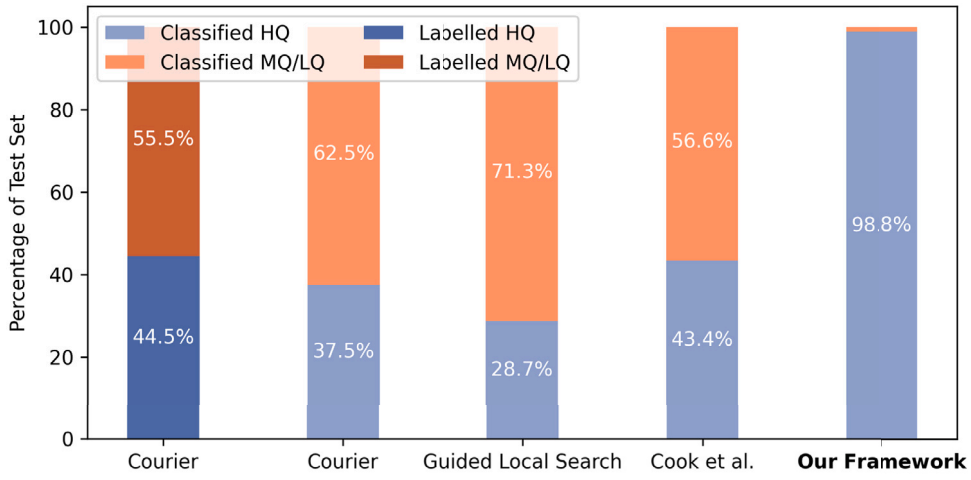
The predictive value correction enhances the reliability of the classifier's output. In Fig. 8(a) shows that 37.5% of the courier-performed routes are *classified* as high quality, while 43.9% of the routes should be expected to have high-quality in the field as shown in Fig. 8(b). The latter is close to the actual labelled data from Amazon, which shows 44.5% of the routes are high-quality. Based on Fig. 8(b), 68.6% of our framework's solutions have a high quality, increasing the number of high-quality solutions by 24.1%, 28.3%, 22.4% compared to the courier-performed routes, travel time-based solutions, and the method by Cook et al. (2022). This highlights that our framework enhances the number of high-quality routes generated despite not using the perfect classifier. Note that the predictive value correction assumes that the classifier's performance generalizes well to unseen data. This assumption is supported by the consistent results observed during cross-validation and testing phases (see Table 3). While these results are promising, there is a trade-off to consider. While our method improves route quality, it does so at the expense of increased travel time. Fig. 9 shows that the median travel time for the framework is 20.6% and 13.9% higher than for the GLS and courier-performed routes, respectively.

In our final evaluation phase, we assessed how closely the routes generated by our framework matched high-quality labelled routes in the test set using the Amazon disparity metric explained in Section 5.2. Table 5 shows that the DIIH performs poorly compared to the benchmark. On average, our prescribed solutions are less similar to the High+Delivered routes compared to the GLS solutions and the winner's method. However, we argue that the disparity metric does not fully represent an algorithm's performance, and our trained classifier may be a better alternative for this purpose. (1) The disparity metric does not take into account the instance structure while evaluating the similarity of two routes. The metric scores algorithms solely on the High+Delivered routes that, based on Section 6.1, have statistically different instance structures than the medium/low-quality routes. The high similarity of solutions to High+Delivered routes, therefore, does not guarantee that an algorithm generates high-quality solutions for medium/low-quality labelled instances. (2) The disparity metric does not predict the quality of a tour in the field. The difference in the disparity scores of two candidate tours does not convey meaningful information about their quality difference. Our model, while not perfect, predicts a route's quality class based on a variety of features, such as the instance structure, visit times to neighbourhoods, and the route's overall structure. (3) For a given instance, multiple high-quality solutions might exist. Thus, the objective should not be to replicate the known high-quality sequences exactly. Deviations from these sequences could represent alternative yet equally viable, high-quality solutions.

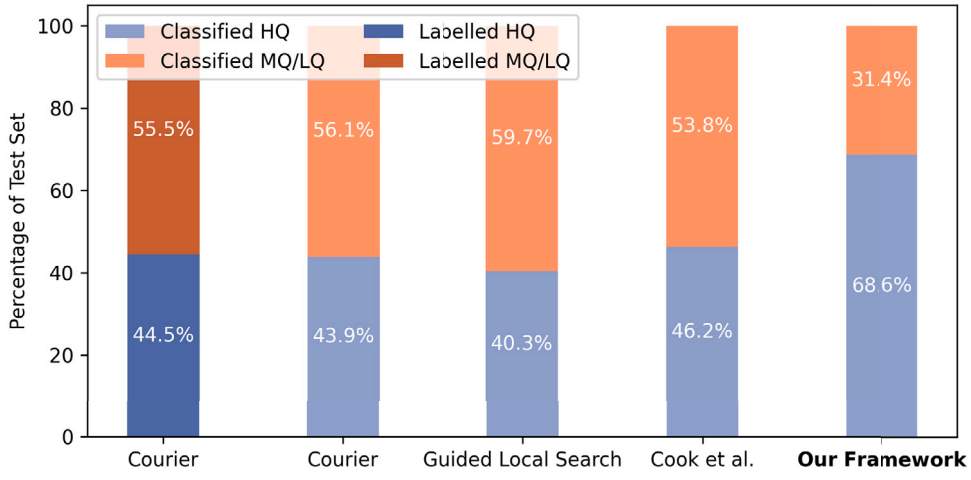
7. Conclusions and future research

Conventional planning algorithms for last-mile deliveries are primarily biased towards the immediate demands of retailers and select consumer groups, while drivers' well-being, safety, and preferences remain unconsidered. This study develops a two-phase framework to treat this algorithmic bias. In the offline phase, the framework trains a Machine Learning classifier on the historical delivery data using supervised learning. The classifier is designed to quantify the probability of a tour's high or medium/low quality in the field. We engineered two sets of features, differing in whether they change as the delivery sequence changes: instance-related (e.g., number of stops, packages, and day type) and route-related features (e.g., route duration or recipient availability likelihood). In the online phase, the framework exploits the insights from the classifier in a data-informed Insertion Heuristic (DIIH) to generate more practical delivery plans. The DIIH dynamically inserts randomly selected unvisited stops into a partial tour. We determine the best insertion position for an unvisited stop by considering the added travel time, the increase in the sharpness of turning angles, backtracking, and the change in the quality of the visit times to the neighbourhoods in the instance. The framework uses the DIIH to create a pool of solutions (i.e., capped to a one-second runtime) and then selects the highest quality solution in the pool using the classifier.

The classifier provides statistical evidence that high-quality routes visit neighbourhoods at different times than when the medium/low-quality routes do; a route with more similar visit times to previously realized high-quality routes has a higher quality. Further, high-quality routes have fewer backtracks to already visited neighbourhoods and are designed to serve more packages compared to medium/low-quality routes. 44.5% of the courier-performed routes in the test set are labelled high-quality by Amazon (i.e., the test set is 15% of the entire dataset). Based on the classifier's *corrected* assessment, the DIIH creates high-quality solutions for 68.6% of the testing set instances (i.e., 24.1% additional high-quality solutions compared to the courier-performed). Note that the trained classifier is not a perfect model, and its assessment is subject to some error. We, however, correct for the classifier's Positive and Negative Predictive Values (PPV and NPV) when reporting the DIIH's performance. This is justified as the classifier shows promising generalizability on unseen data (i.e., the PPV and NPV scores are similar during training and testing). The increase in the number of high-quality solutions comes with a degree of trade-off with total travel time. The median travel time of the



(a) Before predictive value correction



(b) After predictive value correction

Fig. 8. Distribution of the high-quality and medium/low-quality solutions in the testing subset.



Fig. 9. Travel time comparison. The median travel time values are shown in the figure.

data-informed routes is 20.6% higher than the median for solutions with near-optimal total travel time. In comparison, the median travel time for the driver-performed routes is 5.9% higher than the solutions with near-optimal total travel time.

The Amazon challenge used a disparity metric to evaluate the competing algorithms; their metric compared a generated route to the high-quality driver-performed route for the same instance and reported a dissimilarity score. A score closer to 0 indicates a lower disparity between the two routes. The competition's winner scored an average of 0.019 dissimilarity on unseen data, whereas our method achieved an average of 0.119. Although our method does not outperform the state-of-the-art using the disparity metric, we contribute to the literature by developing a learning framework. We learn promising route structures from the high-quality routes while also learning what to avoid from the medium/low-quality routes. Other methods only exploit high-quality routes to generate routes which are not representative of the available dataset. Further, we discuss why the disparity metric used in the Amazon research challenge may not fully represent an algorithm's performance and propose our trained classifier may be a better alternative. Our results suggest that instance-related features affect route quality. Amazon's disparity metric scores algorithms solely on the high-quality routes with a statistically different instance structure than the medium/low-quality routes. Moreover, the disparity metric does not classify Travelling Salesman Problem solutions; it reports a similarity score that does not specify how good of quality a route is. Our classifier predicts a route's quality class based on the instance structure, visit times to neighbourhoods, and the route's overall structure.

While the framework yields promising results, there are opportunities for further research. A key area of focus is the classifier used in the framework; it currently acts as a bottleneck, affecting the performance and parameters of the DIIH. The biases inherent in the classifier can significantly influence the effectiveness of the heuristics. Improving the accuracy of the classifier would likely lead to enhanced outcomes. Future studies should prioritize refining the classifier to result in more accurate and unbiased predictions, improving the overall performance of the route optimization process.

As explained in Section 4.3.1, this study assumes independence between the two components used to estimate the quality of visiting a neighbourhood at a specific time: (i) the frequency with which that time slot appears in historical high-quality routes, and (ii) the success rate of deliveries during that time slot. Future work could explore potential correlations between these components, as high delivery success rates can contribute to route quality, depending on how quality is defined. In contexts where delivery outcomes influence route evaluations more directly, modelling the joint distribution may be appropriate, possibly through sequential or generative approaches that capture such dependencies.

A potential direction for future research is to reformulate the problem as a multi-objective optimization task to explicitly capture the trade-offs among the components of the cost function introduced in Section 4.4.2. Such a formulation would allow for the exploration of Pareto-efficient solutions that balance operational efficiency with route consistency, geometric smoothness, and temporal suitability, rather than relying on a fixed set of scalarized weights.

Another area of research may be to study how driver non-conformance affects other stakeholders of the service (i.e., consumers and the retailer). For instance, imposing delivery time windows for customers is inherently challenging because of fluctuating travel times or weather-induced congestion patterns. This uncertainty is further amplified when drivers deviate from planned routes, leading businesses to potentially overextend delivery windows to ensure customers are available for receipt, which reduces consumer satisfaction. While some deviations from delivery plans improve service, others may lead to suboptimal performance due to drivers' inaccurate or incomplete information. Recognizing that a driver may deviate from a prescribed plan—regardless of the plan's practicality level—strategies should be implemented to minimize the negative impact on other stakeholders.

We also acknowledge the value of benchmarking human-centric delivery systems against emerging alternatives, such as drone-based or autonomous delivery networks. While our study focused on routing within a conventional truck-based fleet, a comparative analysis with autonomous systems could reveal trade-offs between human-operated and fully automated systems, particularly in terms of efficiency, adaptability, and stakeholder satisfaction.

CRediT authorship contribution statement

Hesam Rashidi: Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mehdi Nourinejad:** Writing – review & editing, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization. **Matthew Roorda:** Writing – review & editing, Supervision, Software, Resources, Project administration, Methodology, Funding acquisition, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was funded by the Natural Sciences and Engineering Research Council of Canada, Canada and supported in part by the Roschlau Graduate Fellowship in Sustainable Urban Mobility, awarded by the Mobility Network at the University of Toronto.

Table A.1
Hyperparameter tuning results for tested models. Selected values indicate choices that resulted in the best model performance, while ranges outline tested parameter spaces.

Model	Hyperparameter	Selected Value	Range
Logistic Regression	L2 Penalty	Yes	Yes, No
	Regularization Strength	0.12	0.001–1.0
	Include Intercept	Yes	Yes, No
	Class Weight Adjustment	Balanced	None, Balanced
Random Forest	Number of Estimators	100	100, 150, 200
	Max Depth of Trees	5	1, 5
	Minimum Samples per Node	2	1, 5, 10
	Minimum Samples per Leaf	1	1, 2, 4
	Class Weight Adjustment	Balanced	None, Balanced
Multi-layer Perceptron	Number of Hidden Layers	2	1, 2, 3
	Hidden Dimensions	8	4, 6, 8, 16
	Activation Function	Tanh	ReLU, Tanh
	Solver	Adam	SGD, Adam
	Regularization Strength	0.001	0.0001, 0.001, 0.01, 0.1
	Learning Rate	0.01	0.0001, 0.001, 0.01, 0.1
	Early Stopping	No	Yes, No
GNN with weighted messages	Convolution Layer Type	GCN	GCN, GraphSAGE, GAT
	Learning Rate	0.01	0.0001, 0.001, 0.01, 0.1
	Weight Decay	0.0001	0.0001, 0.001, 0.01, 0.1
	Hidden Dimensions	16	4, 8, 16, 32
	Pooling Function	Sum	Sum, Mean, Max

Appendix A. Hyperparameter tuning results

Table A.1 presents the results of hyperparameter tuning for the tested models. We tested 100 random combinations from the parameter sets listed, selecting values that achieved the best balance of accuracy and generalization during cross-validation. Each parameter is discussed in detail in Section 4.3.4.

Appendix B. Feature importance analysis

To further validate the proposed logistic regression classifier in Section 6.1 and assess the importance of individual features, we applied a permutation-based feature importance testing method, inspired by the approach outlined in Gattermann-Itschert et al. (2023). Permutation-based feature importance testing is a model-agnostic technique that measures the contribution of each feature to model performance by randomly shuffling the feature's values. This shuffling disrupts the relationship between the feature and the dependent variable to quantify the importance of each feature based on the subsequent change in model performance. This method complements the statistical significance measures inherent in logistic regression.

We conducted the permutation-based importance analysis using stratified five-fold cross-validation. We permuted one feature at a time and measured model performance across the five folds with the Area Under the Receiver Operating Characteristic Curve (AUC). The AUC metric is well-suited for this analysis, as it captures the model's ability to discriminate between classes across various classification thresholds.

Fig. B.1 presents a boxplot of permutation effects for each feature across the five cross-validation folds. Each dot represents the AUC score from a single fold. The red vertical line indicates the mean AUC of the classifier with non-permuted variables, serving as a baseline (see Table 3). Boxplots with a median closer to this baseline indicate a smaller effect of permutation, suggesting lower feature importance. Boxplots to the right of the baseline imply that permuting the feature led to improved performance, potentially indicating noise or redundancy in the feature. Conversely, boxplots to the left suggest that permuting the feature decreased performance, underscoring its importance to the model.

Fig. B.1 demonstrates that the median AUC score decreases for all features when permuted, indicating that the feature information is not random. The three most important features are route-related variables that vary with the sequence of stops in the route: bendingEnergy ($r = 9$), backtrackDistance ($r = 9$), and visitTimeQualMLQ ($r = 7, h = 0.5$) < -259. Among the instance-related features, numStops stood out as the most influential. The importance of this feature likely arises from its role in normalizing route characteristics, allowing the model to make better-informed decisions based on the relative complexity and scale of each routing instance.

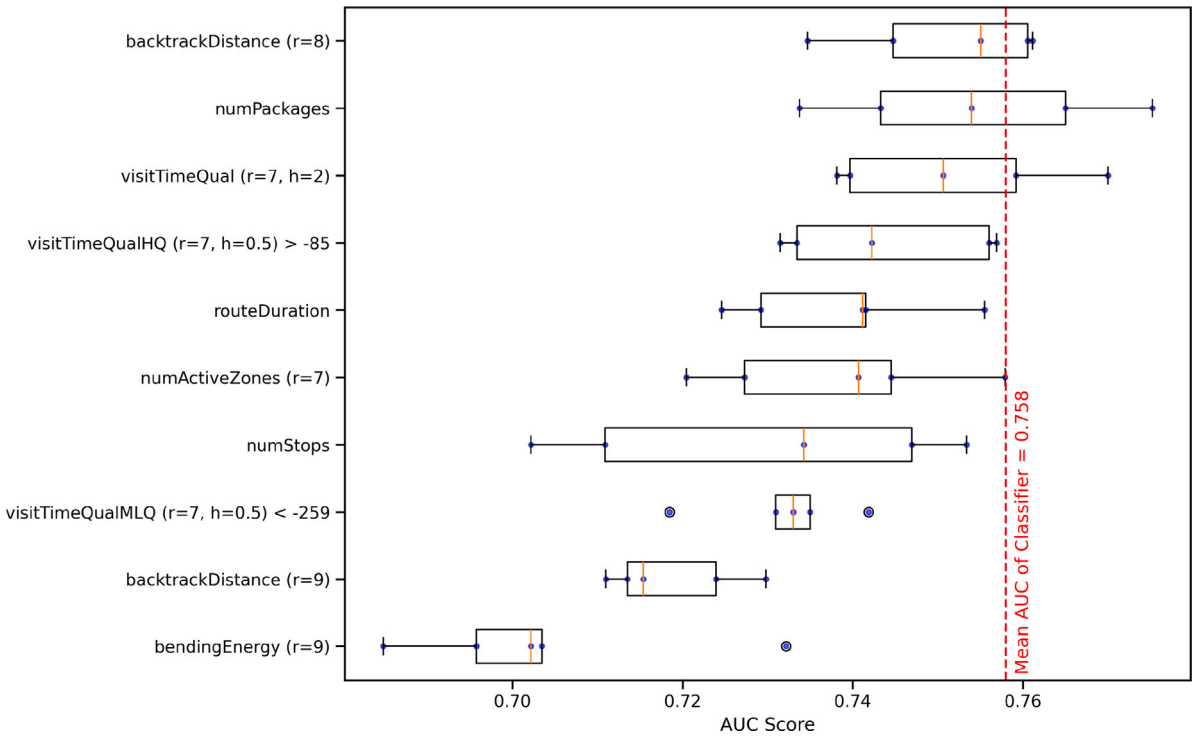


Fig. B.1. Permutation-based feature importance results. Each boxplot shows the distribution of AUC scores across cross-validation folds when a feature is randomly permuted. The dashed vertical line indicates the mean AUC score of the selected classifier with non-permuted features.

Data availability

Data will be made available on request.

References

Bai, R., Chen, X., Chen, Z.L., Cui, T., Gong, S., He, W., Jiang, X., Jin, H., Jin, J., Kendall, G., Li, J., Lu, Z., Ren, J., Weng, P., Xue, N., Zhang, H., 2021. Analytics and machine learning in vehicle routing research. Int. J. Prod. Res. (ISSN: 1366588X) 61, 4–30. http://dx.doi.org/10.1080/00207543.2021.2013566.

Bärmann, A., Martin, A., Pokutta, S., Schneider, O., 2018. An online-learning approach to inverse optimization. http://dx.doi.org/10.48550/arxiv.1810.12997, URL https://arxiv.org/abs/1810.12997v2.

Ben-Akiva, M.E., Ramming, M.S., Bekhor, S., 2004. Route choice models. In: Schreckenberg, M., Selten, R. (Eds.), Human Behaviour and Traffic Networks. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 23–45.

Breiman, L., 2001. Random forests. Mach. Learn. (ISSN: 08856125) 45, http://dx.doi.org/10.1023/A:1010933404324.

Campbell, A.M., Savelsbergh, M., 2004. Efficient insertion heuristics for vehicle routing and scheduling problems. Transp. Sci. (ISSN: 00411655) 38, http://dx.doi.org/10.1287/trsc.1030.0046.

Canoy, R., Bucarey, V., Mandi, J., Guns, T., 2023. Learn and route: learning implicit preferences for vehicle routing. Constraints (ISSN: 15729354) 28, 363–396. http://dx.doi.org/10.1007/s10601-023-09363-2, URL https://link.springer.com/article/10.1007/s10601-023-09363-2.

Canoy, R., Bucarey, V., Mandi, J., Mulamba, M., Molenbruch, Y., Guns, T., 2024. Probability estimation and structured output prediction for learning preferences in last-mile delivery. Comput. Ind. Eng. (ISSN: 0360-8352) 189, 109932. http://dx.doi.org/10.1016/J.CIE.2024.109932.

Canoy, R., Guns, T., 2019. Vehicle routing by learning from historical solutions. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics, 11802 LNCS, Springer, (ISSN: 16113349) pp. 54–70. http://dx.doi.org/10.1007/978-3-030-30048-7_4, URL https://link.springer.com/chapter/10.1007/978-3-030-30048-7_4.

Chan, T.C., Lee, T., Terekhov, D., 2018. Inverse optimization: Closed-form solutions, geometry, and goodness of fit. (ISSN: 15265501) 65, 1115–1135. http://dx.doi.org/10.1287/MNSC.2017.2992, URL https://pubsonline.informs.org/doi/abs/10.1287/mnsc.2017.2992.

Chen, L., Chen, Y., Langevin, A., 2021. An inverse optimization approach for a capacitated vehicle routing problem. European J. Oper. Res. (ISSN: 03772217) 295, http://dx.doi.org/10.1016/j.ejor.2021.03.031.

Cook, W., Held, S., Helsgaun, K., 2022. Constrained local search for last-mile routing. Transp. Sci. (ISSN: 0041-1655) http://dx.doi.org/10.1287/TRSC.2022.1185, URL https://pubsonline.informs.org/doi/abs/10.1287/trsc.2022.1185.

Dantzig, G.B., Ramser, J.H., 1959. The truck dispatching problem. Manag. Sci. (ISSN: 0025-1909) 6, http://dx.doi.org/10.1287/mnsc.6.1.80.

Dia, H., 2002. An agent-based approach to modelling driver route choice behaviour under the influence of real-time information. Transp. Res. Part C: Emerg. Technol. (ISSN: 0968-090X) 10 (5), 331–349. http://dx.doi.org/10.1016/S0968-090X(02)00025-6, URL https://www.sciencedirect.com/science/article/pii/S0968090X02000256.

Dieter, P., Caron, M., Schryen, G., 2023. Integrating driver behavior into last-mile delivery routing: combining machine learning and optimization in a hybrid decision support framework. European J. Oper. Res. (ISSN: 0377-2217) 311, 283–300. http://dx.doi.org/10.1016/J.EJOR.2023.04.043.
Dietterich, T.G., 1998. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Comput. (ISSN: 08997667) 10, http://dx.doi.org/10.1162/089976698300017197.
Florio, A.M., Feillet, D., Hartl, R.F., 2018. The delivery problem: optimizing hit rates in e-commerce deliveries. Transp. Res. Part B: Methodol. (ISSN: 0191-2615) 117, 455–472. http://dx.doi.org/10.1016/J.TRB.2018.09.011.
Gattermann-Itschert, T., Poreschack, L.M., Thonemann, U.W., 2023. Using machine learning to include planners' preferences in railway crew scheduling optimization. Transp. Sci. (ISSN: 0041-1655) 57, 796–812. http://dx.doi.org/10.1287/TRSC.2022.1196, URL https://pubsonline.informs.org/doi/abs/10.1287/trsc.2022.1196.
Gretton, C., Kilby, P., 2013. A study of shape penalties in vehicle routing. URL https://tristanconference.org/system/tristan-viii/book_of_abstracts/data/Routing/TRISTAN8_paper_6.pdf.
Hamilton, W.L., 2020. Graph representation learning. Synth. Lect. Artif. Intell. Mach. Learn. 14, 1–159. http://dx.doi.org/10.1007/978-3-031-01588-5.
Hamilton, W.L., Ying, R., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems. (ISSN: 10495258) 2017-December.
Holland, C., Levis, J., Nuggehalli, R., Santilli, B., Winters, J., 2017. UPS optimizes delivery routes. Interfaces (ISSN: 1526551X) 47, 8–23. http://dx.doi.org/10.1287/inte.2016.0875, URL https://pubsonline.informs.org/doi/abs/10.1287/inte.2016.0875.
Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. In: 5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings.
Kovacs, A.A., Golden, B.L., Hartl, R.F., Parragh, S.N., 2015a. The generalized consistent vehicle routing problem. Transp. Sci. (ISSN: 15265447) 49, 796–816. http://dx.doi.org/10.1287/TRSC.2014.0529, URL http://pubsonline.informs.org816 http://www.informs.org.
Kovacs, A.A., Parragh, S.N., Hartl, R.F., 2015b. The multi-objective generalized consistent vehicle routing problem. European J. Oper. Res. (ISSN: 0377-2217) 247, 441–458. http://dx.doi.org/10.1016/J.EJOR.2015.06.030.
Krumm, J., 2008. A Markov model for driver turn prediction. URL https://www.microsoft.com/en-us/research/publication/markov-model-driver-turn-prediction/,
Li, Y., Phillips, W., 2018. Learning from route plan deviation in last-mile delivery. https://dspace.mit.edu/handle/1721.1/118135.
Liu, S., He, L., Shen, Z.J.M., 2021. On-time last-mile delivery: Order assignment with travel-time predictors. Manag. Sci. (ISSN: 15265501) 67, http://dx.doi.org/10.1287/mnsc.2020.3741.
Luo, Z., Qin, H., Che, C., Lim, A., 2015. On service consistency in multi-period vehicle routing. European J. Oper. Res. (ISSN: 0377-2217) 243, 731–744. http://dx.doi.org/10.1016/J.EJOR.2014.12.019.
Mccoy, R.T., Frank, R., 2018. Phonologically informed edit distance algorithms for word alignment with low-resource languages. Proc. Soc. Comput. Linguist. 1, 102–112. http://dx.doi.org/10.7275/R5251GC0.
Merchán, D., Arora, J., Pachon, J., Konduri, K., Winkenbach, M., Parks, S., Noszek, J., Merchán, M., 2022. Amazon last mile routing research challenge: Data set. Transp. Sci. http://dx.doi.org/10.1287/trsc.2022.1173, URL http://pubsonline.informs.orghttp://www.informs.orghttps://orcid.org/0000-0002-8018-9565.
MIT CAVE lab, 2021. The amazon challenge's scoring function. URL https://github.com/MIT-CAVE/rc-cli/tree/main/scoring.
Nourmohammadi, Z., Hu, B., Rey, D., Saberi, M., 2025. A data-driven preference learning approach for multi-objective vehicle routing problems in last-mile delivery. Transp. Res. Part C: Emerg. Technol. 174, 105101.
Özarık, S.S., da Costa, P., Florio, A.M., 2023. Machine learning for data-driven last-mile delivery optimization. Transp. Sci. (ISSN: 15265447) 58, 27–44. http://dx.doi.org/10.1287/trsc.2022.0029, URL https://pubsonline.informs.org/doi/abs/10.1287/trsc.2022.0029.
Özarık, S., Veelenturf, L.P., Woensel, T.V., Laporte, G., 2021. Optimizing e-commerce last-mile vehicle routing and scheduling under uncertain customer presence. Transp. Res. Part E: Logist. Transp. Rev. (ISSN: 1366-5545) 148, 102263. http://dx.doi.org/10.1016/J.TRE.2021.102263.
Peng, C.Y.J., Lee, K.L., Ingersoll, G.M., 2002. An introduction to logistic regression analysis and reporting. J. Educ. Res. (ISSN: 19400675) 96, http://dx.doi.org/10.1080/00220670209598786.
Perron, L., Furnon, V., 2023. OR-tools. URL https://developers.google.com/optimization.
Poot, A., Kant, G., Wagelmans, A.P., 2002. A savings based method for real-life vehicle routing problems. J. Oper. Res. Soc. (ISSN: 14769360) 53, http://dx.doi.org/10.1057/palgrave/jors/2601252.
Python, 2024. Data structures — Python 3.12.2 documentation. URL https://docs.python.org/3/tutorial/datastructures.html#dictionaries.
Quirion-Blais, O., Chen, L., 2021. A case-based reasoning approach to solve the vehicle routing problem with time windows and drivers' experience. Omega (United Kingdom) (ISSN: 03050483) 102, http://dx.doi.org/10.1016/j.omega.2020.102340.
Rastogi, N., 2023. Embracing a people-centric design in last-mile delivery and elsewhere. URL https://www.forbes.com/sites/forbesbusinesscouncil/2023/08/03/embracing-a-people-centric-design-in-last-mile-delivery-and-elsewhere/?sh=171d547c8e65.
Reyes, D., Savelsbergh, M., Toriello, A., 2017. Vehicle routing with roaming delivery locations. Transp. Res. Part C: Emerg. Technol. (ISSN: 0968-090X) 80, 71–91. http://dx.doi.org/10.1016/j.trc.2017.04.003, URL https://www.sciencedirect.com/science/article/pii/S0968090X17301092.
Rochat, Y., Taillard, É.D., 1995. Probabilistic diversification and intensification in local search for vehicle routing. J. Heuristics (ISSN: 13811231) 1, 147–167. http://dx.doi.org/10.1007/BF02430370, URL https://link.springer.com/article/10.1007/BF02430370.
Rosenkrantz, D.J., Stearns, R.E., Philip M. Lewis, I., 1977. An analysis of several heuristics for the traveling salesman problem. SIAM J. Comput. (ISSN: 0097-5397) 6, http://dx.doi.org/10.1137/0206041.
Rossit, D.G., Vigo, D., Tohmé, F., Frutos, M., 2019. Visual attractiveness in routing problems: A review. Comput. Oper. Res. (ISSN: 03050548) 103, http://dx.doi.org/10.1016/j.cor.2018.10.012.
Schneider, M., 2016. The vehicle routing problem with time windows and driver-specific times. European J. Oper. Res. (ISSN: 0377-2217) 250, 101–119. http://dx.doi.org/10.1016/J.EJOR.2015.09.015.
Schober, P., Schwarte, L.A., 2018. Correlation coefficients: Appropriate use and interpretation. Anesth. Analg. (ISSN: 15267598) 126, 1763–1768. http://dx.doi.org/10.1213/ANE.0000000000002864.
Sun, H.J.N., Arslan, O., 2025. A contextual framework for learning routing experiences in last-mile delivery. Transp. Res. Part B: Methodol. 194, 103172.
Sun, J., Zhang, D.J., Hu, H., Mieghem, J.A.V., 2022. Predicting human discretion to adjust algorithmic prescription: A large-scale field experiment in warehouse operations. Manag. Sci. (ISSN: 15265501) 68, 846–865. http://dx.doi.org/10.1287/mnsc.2021.3990.
Uber, 2023. Hexagonal hierarchical geospatial indexing system. URL https://h3geo.org/.
Ulmer, M., Nowak, M., Mattfeld, D., Kaminski, B., 2020. Binary driver-customer familiarity in service routing. European J. Oper. Res. (ISSN: 0377-2217) 286, 477–493. http://dx.doi.org/10.1016/J.EJOR.2020.03.037.
Veličković, P., Casanova, A., Liò, P., Cucurull, G., Romero, A., Bengio, Y., 2017. Graph attention networks. In: 6th International Conference on Learning Representations, ICLR 2018 - Conference Track Proceedings. International Conference on Learning Representations, ICLR, http://dx.doi.org/10.1007/978-3-031-01587-8_7, URL https://arxiv.org/abs/1710.10903v3.
VeRoViz, 2023. VeRoViz - vehicle routing visualization. URL https://veroviz.org/.
Vickers, D., Lee, M.D., Dry, M., Hughes, P., McMahon, J.A., 2006. The aesthetic appeal of minimal structures: Judging the attractiveness of solutions to traveling salesperson problems. Percept. Psychophys. (ISSN: 00315117) 68, http://dx.doi.org/10.3758/BF03193653.

White, B.W., Rosenblatt, F., 1963. Principles of neurodynamics: Perceptrons and the theory of brain mechanisms. Am. J. Psychol. (ISSN: 00029556) 76, http://dx.doi.org/10.2307/1419730.

Winkenbach, M., Spinler, S., Pachon, J., Konduri, K., 2024. Introduction to the special issue on machine learning methods and applications in large-scale route planning problems. Transp. Sci. (ISSN: 15265447) 58, http://dx.doi.org/10.1287/trsc.2023.intro.v58.n1.

Ye, N., Wang, Z.Q., Malekian, R., Lin, Q., Wang, R.C., 2015. A method for driving route predictions based on hidden Markov model. Math. Probl. Eng. (ISSN: 15635147) 2015, http://dx.doi.org/10.1155/2015/824532.

Zattoni Scroccaro, P., van Beek, P., Mohajerin Esfahani, P., Atasoy, B., 2025. Inverse optimization for routing problems. Transp. Sci. 59 (2), 301–321.

Zhou, F., He, Y., Zhou, L., 2019. Last mile delivery with stochastic travel times considering dual services. IEEE Access (ISSN: 21693536) 7, http://dx.doi.org/10.1109/ACCESS.2019.2950442.